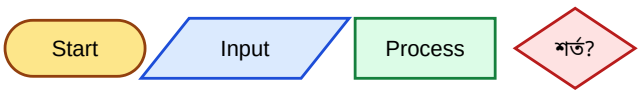
০ কীভাবে পড়বে + সূচিপত্র

পরীক্ষার রাতের প্ল্যান

রঙের মানে

নীল বক্স	সংজ্ঞা — হুবহু লিখতে হবে
কমলা বক্স	পরীক্ষায় বেশি আসে (নোটে *** দেওয়া)
সবুজ বক্স	শর্টকাট / টিপস
বেগুনি বক্স	মনে রাখার কৌশল
লাল বক্স	সাধারণ ভুল / ফাঁদ
⚠ ড্যাশ বক্স	নোটের ভুলের সংশোধন
কালো বক্স	C কোড; পাশে কালো টার্মিনালে আসল আউটপুট

ফ্লোচার্টের রং



পরীক্ষার রাতের ৩ ধাপ

1. **১ম পাঠ (৯০ মি.):** অংশ ১–১০; অ্যালগরিদম–ফ্লোচার্ট–কোড খাতায় একবার নিজে লেখো।
2. **২য় পাঠ (২০ মি.):** শুধু রঙিন বক্স ও সংশোধনগুলো।
3. **শেষ ১০ মি.:** MCQ + এক পৃষ্ঠার রিভিশন।

সূচিপত্র

অংশ	বিষয়
১	প্রোগ্রাম ও প্রোগ্রামিং ভাষা
২	সোর্স কোড, অবজেক্ট কোড; মেশিন বনাম উচ্চস্তরের ভাষা
২ক	কম্পিউটার সংগঠন, প্রোগ্রামের ভুল, অ্যাসেম্বলি, মিড লেভেল **(নতুন)**
৩	প্রোগ্রামিং ভাষার ৫ প্রজন্ম
৪	অনুবাদক প্রোগ্রাম
৫	প্রোগ্রাম তৈরির ৭ ধাপ
৬	অ্যালগরিদম
৭	ফ্লোচার্ট: প্রতীক ও মৌলিক ছাঁচ
৮	সমাধানকৃত A-F-C (১২টি)
৮ক	আরও ১০টি: গড়, F→C, হেরন, বৃত্ত, জোড়-বিজোড়, অক্ষর, Leap Year, GCD, Factorial **(নতুন)**
৮খ	সিরিজ-সমষ্টি মাস্টার টেবিল, goto-স্টাইল, ট্রেস টেবিল **(নতুন)**
৯	C: ইতিহাস, বৈশিষ্ট্য, গঠন, হেডার, main
৯ক	ডেটা টাইপ, চলক, ধ্রুবক
৯খ	scanf, printf, Escape sequence, Comment
৯গ	অপারেটর ও এক্সপ্রেশন
৯ঘ	if-else, ladder, nested, switch
৯ঙ	লুপ, break, continue
৯চ	ফাংশন, অ্যারে, স্ট্রিং
১০	নোট-২-এর A-F-C: যোগ-বিয়োগ, গড়, তাপমাত্রা, ত্রিভুজ
১১	শর্টকাট, সাধারণ ভুল ও সব সংশোধন
১২	৩৪টি MCQ + ৪টি সৃজনশীল
১৩	এক পৃষ্ঠার শেষ মুহূর্তের রিভিশন

১ প্রোগ্রাম ও প্রোগ্রামিং ভাষা

মৌলিক ধারণা

প্রোগ্রাম (Program) নোট-১ + নোট-২

- যন্ত্রের মাধ্যমে কোনো সমস্যা সমাধানের লক্ষ্যে প্রোগ্রামিং ভাষা ব্যবহার করে প্রোগ্রামারের দেওয়া **ধারাবাহিক নির্দেশনার সমষ্টিকে** প্রোগ্রাম বলে। নোট-১
- কোনো সমস্যা সমাধানের জন্য Computer-এর ভাষায় লিখিত ধারাবাহিক কতগুলো Command বা নির্দেশের সমষ্টিকে Program বলে। নোট-২
- *A program is a set of instructions written in a programming language that a computer can execute to perform a specific task.*

প্রোগ্রামিং ভাষা (Programming Language)

নোট-১ + নোট-২

- যে সকল সংকেত বা নিয়ম মেনে কম্পিউটারকে কোনো আদেশ বা নির্দেশ প্রদান করে কোনো কাজ করানো হয় তাই programming language। নোট-১
- যে ভাষা ব্যবহার করে Program লেখা হয় তাকে Programming Language বলে। নোট-২

উদাহরণ: C, C++, C# (C sharp), Java, Python, Basic, Fortran, Cobol …

শব্দের অর্থ নোট-২

Program = কার্যক্রম (কাজ) • Language = ভাষা (ধারাবাহিকতা)

অর্থাৎ **কাজের ধারাবাহিকতা অনুযায়ী** প্রোগ্রামের কাজ করতে হবে।

Program-এর অপর নাম: **Software, Application (App)**।

🧠 সফটওয়্যার = ভবন নোট-২

ভবন তৈরি	প্রোগ্রাম তৈরি
① পরিকল্পনা করতে হবে	① **Algorithm** (অ্যালগরিদম) — লিখিত পরিকল্পনা
② ম্যাপ আঁকতে হবে	② **Flow-chart** (প্রবাহচিত্র) — চিত্র/নকশা
③ বাস্তবায়ন করতে হবে	③ **C-Program Coding** — আসল নির্মাণ

মনে রাখো: **"A → F → C"** — পরীক্ষায় "A, F ও C লেখ" মানে এই তিনটি।

২ সোর্স কোড, অবজেক্ট কোড; মেশিন বনাম উচ্চস্তরের ভাষা

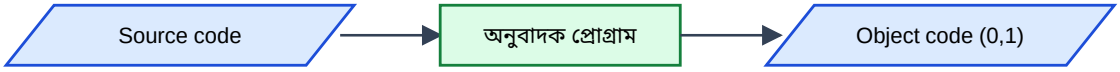
মেশিন ভাষা — কম্পিউটারের ভাষা নোট-১

শুধু 0 ও 1 দিয়ে লেখা:

```
01001010101
10110010101
10001110100
```

কম্পিউটারের ভেতরে **0 = 0 volt** (বিদ্যুৎ নেই), **1 = 5 volt** (বিদ্যুৎ আছে)।

শব্দ নোট-১	অর্থ
Source file / Source code	programming language দিয়ে লেখা file (উৎস প্রোগ্রাম)
Machine code / Object code	Source file-কে রূপান্তরের পর যে code পাওয়া যায় [0,1] (বস্তু প্রোগ্রাম)
Compiler	Source file-কে machine code-এ রূপান্তরের কাজ যে করে



মেশিন ভাষা বনাম উচ্চস্তরের ভাষা (৮ পয়েন্ট) ** নোট-১

#	Machine Language [0,1]	High Level Language
১	কম্পিউটার যে ভাষা সরাসরি বুঝতে পারে তাকে machine language বলে।	কম্পিউটারকে সর্বজনে ব্যবহার উপযোগী করে তুলতে যে ভাষা তৈরি হয় তাই High level language।
২	এটি মেশিনের ভাষা বিধায় কম্পিউটারকে বুঝানোর জন্য অনুবাদের প্রয়োজন হয় না।	এ ভাষায় লিখিত প্রোগ্রাম কম্পিউটারকে বোঝানোর জন্য মেশিনের ভাষায় অনুবাদ করে দিতে হয়।
৩	প্রোগ্রাম লেখা তুলনামূলকভাবে কষ্টসাধ্য এবং সময়সাপেক্ষ।	মেশিন ভাষার তুলনায় প্রোগ্রাম রচনা সহজ।
৪	লিখিত প্রোগ্রামকে **অবজেক্ট প্রোগ্রাম বা বস্তু প্রোগ্রাম** বলে।	লিখিত প্রোগ্রামকে **Source বা উৎস প্রোগ্রাম** বলে।
৫	বাইনারি, যা 0 এবং 1 দ্বারা গঠিত।	উদাহরণ: C++, BASIC, PASCAL, FORTRAN ইত্যাদি।
৬	মেশিন সহজে বুঝতে পারে।	মেশিন সহজে (সরাসরি) বুঝতে পারে না।
৭	ভুল হওয়ার সম্ভাবনা বেশি থাকে।	ভুল হওয়ার সম্ভাবনা কম থাকে।
৮	প্রোগ্রাম ডিবাগ করা কঠিন।	প্রোগ্রাম ডিবাগ করা অপেক্ষাকৃত সহজ।

🧠

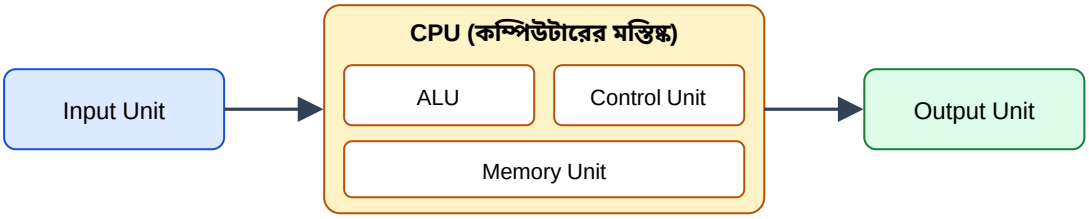
মেশিন = Object = বস্তু = 0/1 • High level = Source = উৎস = ইংরেজির মতো। "উৎস থেকে বস্তু বানায় অনুবাদক।"

২ক কম্পিউটার সংগঠন + ভাষার বাড়তি তথ্য

নতুন PDF (ফাইনাল নোট) থেকে যোগ হলো

ক) কম্পিউটার সংগঠন — সংক্ষেপে

ফাইনাল নোট



অংশ	কাজ
ইনপুট অংশ	কম্পিউটারকে নির্দেশ/ডেটা দেওয়ার যন্ত্র। যেমন — কীবোর্ড, মাউস, স্ক্যানার, ওয়েবক্যাম, OCR, লাইটপেন, কার্ড রিডার, ডিজিটাল ক্যামেরা।
আউটপুট অংশ	প্রক্রিয়াজাত ফলাফল ব্যবহারকারীর বোধগম্য রূপে বাইরে প্রকাশ করে (মনিটর, প্রিন্টার, স্পিকার)।
CPU	সবচেয়ে গুরুত্বপূর্ণ অংশ, কম্পিউটারের **মস্তিষ্ক**। ডেটা প্রক্রিয়াজাত করে আউটপুটে রূপান্তর করে; গাণিতিক হিসাব, যৌক্তিক সিদ্ধান্ত ও পুরো সিস্টেম নিয়ন্ত্রণ করে। **৩টি অংশ:**
১. ALU	Arithmetic and Logic Unit — যোগ, বিয়োগ, গুণ, ভাগ এবং তুলনা (>, <, =); AND, OR, NOT দিয়ে সত্য/মিথ্যা নির্ণয়।
২. Control Unit	নির্দেশ পরীক্ষা করে সংকেত তৈরি করে; কখন ALU কাজ করবে, কখন মেমোরি থেকে ডেটা আসবে — তা নিয়ন্ত্রণ করে।
৩. Memory Unit	তথ্য জমা রাখে; কাজের সময় প্রয়োজনমতো এখান থেকে তথ্য নেওয়া হয়।

মেমোরি	বৈশিষ্ট্য	উদাহরণ
Volatile (উদ্বায়ী)	বিদ্যুৎ গেলে ডেটা মুছে যায়; সাময়িক কাজের জন্য; খুব দ্রুত	RAM
Non-volatile (অ-উদ্বায়ী)	বিদ্যুৎ গেলেও ডেটা থাকে; স্থায়ী সংরক্ষণ; তুলনামূলক ধীর	ROM, HDD, **SSD**, DVD, USB Drive

🧠 **মনে রাখো**

"ভোলা RAM, ভোলে না ROM" — RAM বিদ্যুৎ গেলে সব ভুলে যায়; ROM/HDD/SSD/USB মনে রাখে। ডেটার ক্ষুদ্রতম একক = **বিট (Bit)**।

খ) প্রোগ্রাম-সংক্রান্ত নতুন সংজ্ঞা

ফাইনাল নোট

প্রোগ্রামিং ও প্রোগ্রামার

প্রোগ্রামিং: প্রোগ্রাম রচনার পদ্ধতি/কৌশল।
প্রোগ্রামার: যিনি প্রোগ্রামিং করেন।
English: A program is a set of instructions written in a programming language that a computer can execute to perform a specific task.

প্রোগ্রামের কাঠামো — ৩ অংশ (IPO)	
Input	ব্যবহারকারীর কাছ থেকে ডেটা নেওয়া
Process	ডেটার উপর গণনা/প্রক্রিয়া
Output	ফলাফল দেখানো

🎯 প্রোগ্রামিং-এর প্রবর্তক

অ্যাডা লাভলেস (Ada Lovelace) — কবি লর্ড বায়রনের কন্যা। চার্লস ব্যাবেজের Analytical Engine-এর জন্য **Bernoulli Number** নির্ণয়ের ধারাবাহিক হিসাব লিখে ইতিহাসের প্রথম প্রোগ্রাম রচনা করেন। (MCQ-তে "প্রথম প্রোগ্রামার" = Ada Lovelace)

প্রোগ্রামের ৩ ধরনের ভুল (Error)

১. Data Error: ভুল ইনপুট/উপাত্ত দিলে।
২. Logical Error: যুক্তিতে ভুল — প্রোগ্রাম চলে কিন্তু ভুল ফল দেয় (যেমন অংশ ১০-এর `(1/2)*b*h`)।
৩. Syntax Error: ভাষার ব্যাকরণে ভুল (; বাদ, viod) — প্রোগ্রাম কম্পাইলই হয় না।

Dry Run

প্রোগ্রাম কম্পিউটারে চালানোর আগে নমুনা ডেটা দিয়ে **হাতে-কলমে** (কম্পিউটার ছাড়া) ধাপগুলো চালিয়ে ঠিক ফল আসে কিনা যাচাই করা। (অংশ ৮ক-এর "ট্রেস টেবিল" আসলে Dry Run।)

গ) মেশিন ভাষা ও অ্যাসেম্বলি — বাড়তি তথ্য

ফাইনাল নোট

মেশিন ভাষায় ৪ ধরনের নির্দেশ	
গাণিতিক (Arithmetic)	যোগ, বিয়োগ, গুণ, ভাগ
নিয়ন্ত্রণ (Control)	Load, Store, Jump
ইনপুট-আউটপুট	Read, Write
প্রত্যক্ষ ব্যবহার (Direct Use)	Start, Halt, End

মেশিন ভাষায় ০ = বিদ্যুৎ OFF, ১ = ON। মেশিন ভাষায় লেখা প্রোগ্রাম = Object Program। সুবিধা: সরাসরি চলে, অনুবাদ লাগে না, দ্রুত, অল্প মেমোরি। অসুবিধা: শেখা/লেখা কঠিন, ভুল বেশি, ডিবাগ কঠিন, **যন্ত্র নির্ভর**।

অ্যাসেম্বলি নির্দেশের ৪ অংশ (উদাহরণ: `LOOP: ADD eax, ebx ; যোগ`)	
Label	নির্দিষ্ট স্থান চিহ্নিত করে; Jump দিয়ে এখানে আসা যায় (LOOP:)
Opcode	কী কাজ হবে — নেমোনিক (ADD)
Operand	যার উপর কাজ — রেজিস্টার/মেমোরি ঠিকানা (eax, ebx)
Comment	ব্যাখ্যা; চলায় কোনো ভূমিকা নেই (; যোগ)

অ্যাসেম্বলিকে **প্রতীকী ভাষা (Symbolic language)**-ও বলে।

⚠ সংশোধন

উৎসে: ফাইনাল নোট (অ্যাসেম্বলির অসুবিধা): "এক লাইন এক লাইন পড়ে অনুবাদ করায় ... প্রতিবার রান করার সময় পুনরায় অনুবাদ প্রয়োজন হয়"; সুবিধা: "আলাদা Object code মেমোরিতে জমা রাখতে হয় না"
সঠিক: Assembler পুরো অ্যাসেম্বলি প্রোগ্রামকে একবারে মেশিন কোডে (Object code) রূপান্তর করে; সেই Object code বারবার চালানো যায়, প্রতিবার অনুবাদ লাগে না। অ্যাসেম্বলির আসল অসুবিধা — **যন্ত্র নির্ভর** (প্রতিটি প্রসেসরের আলাদা অ্যাসেম্বলি), বড় প্রোগ্রাম লেখা কষ্টসাধ্য।
কেন: "লাইন-বাই-লাইন, প্রতিবার অনুবাদ, Object code নেই" — এগুলো **Interpreter**-এর বৈশিষ্ট্য; ভুল করে অ্যাসেম্বলির ঘরে চলে গেছে। পরীক্ষায় এভাবে লিখলে নম্বর কাটবে।

ঘ) উচ্চস্তরের ভাষার ব্যবহারক্ষেত্র ও মিড লেভেল ভাষা

ফাইনাল নোট

উচ্চস্তরের ভাষার ব্যবহারক্ষেত্র

১) বড় প্রোগ্রাম তৈরিতে ২) বৃহৎ ডেটা প্রসেসিং ৩) জটিল গাণিতিক সফটওয়্যার ৪) Automatic Process Control ৫) Application Package Software।
অতিরিক্ত অসুবিধা: রেজিস্টার সরাসরি নিয়ন্ত্রণ করা যায় না; লেখার আগে ভাষার নিয়ম জানা আবশ্যক।

মিড লেভেল ল্যাঙ্গুয়েজ (Mid Level Language)

যে ভাষায় নিম্নস্তর ও উচ্চস্তর — দুই ধরনের সুবিধাই আছে: হার্ডওয়্যারের বিট/বাইট পর্যায়ে নিয়ন্ত্রণ (নিম্নস্তরের গুণ) এবং এক মেশিনের প্রোগ্রাম অন্য মেশিনে চলে — Portable (উচ্চস্তরের গুণ)। **সবচেয়ে প্রচলিত উদাহরণ: C**।

🧠 C কেন মিড লেভেল?

"**নিচে বিট-বাইট ছোঁয়, উপরে যেকোনো মেশিনে দৌড়ায়**" — বিটওয়াইজ অপারেটর (<<, >>, &) দিয়ে হার্ডওয়্যারের কাছাকাছি কাজ, আবার Portable।

4GL ও 5GL-এর বাড়তি উদাহরণ

4GL (Very High Level, ডেটাবেজ-অভিমুখী): SQL, Visual Basic, Oracle, NOMAD, Focus।

5GL (Natural Language, AI): PROLOG, LISP, Mercury, OPS5।

সময়কাল (প্রচলিত): 1GL 1945–50, 2GL 1950–60, 3GL 1960–70, 4GL 1970–80, 5GL 1980–বর্তমান।

⚠ সংশোধন

উৎসে: ফাইনাল নোট: মিড লেভেলের উদাহরণ "FORTRAN, COBOL, Macro Assembler, C"

সঠিক: মিড লেভেলের নির্ভরযোগ্য উদাহরণ **C** (কখনো C++ ও Macro Assembler-ও বলা হয়)। **FORTRAN ও COBOL হলো উচ্চস্তরের (3GL) ভাষা**।

কেন: এই নোটেরই অংশ ৩-এ FORTRAN, COBOL উচ্চস্তরের ভাষার উদাহরণ। এগুলোতে বিট-লেভেলে হার্ডওয়্যার নিয়ন্ত্রণ নেই — তাই মিড লেভেল বলা ঠিক নয়। MCQ-তে "মিড লেভেল ভাষা কোনটি?" এলে উত্তর C।

⚠ সংশোধন

উৎসে: ফাইনাল নোট: Interpreter-এর উদাহরণ "Python, Pascal, Basic"

সঠিক: Interpreter: **Python, BASIC** (এছাড়া JavaScript, PHP)। **Pascal সাধারণত Compiler দিয়ে চলে**।

কেন: Pascal, C, C++, FORTRAN — কম্পাইলড ভাষা। ফাইনাল নোটের নিজের তুলনা-সারণিতেও "Pascal (কম্পাইলড ব্যবহারে)" লেখা আছে — দুই জায়গায় অমিল।

৩ প্রোগ্রামিং ভাষার ৫টি প্রজন্ম

প্রোগ্রামিং-এর ইতিহাস নোট-১

1945 থেকে শুরু করে এ পর্যন্ত যত প্রোগ্রামিং ভাষা আবিষ্কৃত হয়েছে তাদেরকে বৈশিষ্ট্য অনুযায়ী **পাঁচটি প্রজন্মে** ভাগ করা হয়েছে।

প্রজন্ম	সময়কাল	ভাষা	উদাহরণ / ব্যবহার
১ম (First Generation)	1945–1950	**Machine Language** (যান্ত্রিক ভাষা)	0 ও 1
২য় (Second)	1950–1960	**Assembly Language** (অ্যাসেম্বলি ভাষা)	নেমোনিক: `add eax, ebx`
৩য় (Third)	1960–1970	**High Level Language** (উচ্চস্তরের ভাষা)	Fortran, Basic, Pascal, Cobol, C, C++, Visual Basic, Java
৪র্থ (Fourth) — 4GL	1970–1980	**Very High Level Language** (অতি উচ্চস্তরের ভাষা)	ডেটাবেজ প্রোগ্রামিং ও স্ক্রিপ্টিং (যেমন SQL) উদাহরণ সংযোজন
৫ম (Fifth) — 5GL	1980–বর্তমান	**Natural Language** (স্বাভাবিক ভাষা)	কৃত্রিম বুদ্ধিমত্তা: PROLOG, LISP, Mercury

🧠 **প্রজন্ম মুখস্থ: "মেশিন-অ্যাসে-হাই-ভেরি-ন্যাচারাল"**

১ম মেশিন → **২য়** অ্যাসেম্বলি → **৩য়** হাই → **৪র্থ** ভেরি হাই → **৫ম** ন্যাচারাল।

সাল: **45-50-60-70-80** — প্রথম ধাপ ৫ বছর, বাকি সব ১০ বছর করে।

⚡ **নিম্নস্তর বনাম উচ্চস্তর** সংযোজন

১ম ও ২য় প্রজন্ম = **নিম্নস্তরের ভাষা** (যন্ত্র নির্ভর)।

৩য় থেকে ৫ম = **উচ্চস্তরের ভাষা** (যন্ত্র নির্ভর নয়)।

প্রজন্ম যত বাড়ে → মানুষের কাছে তত সহজ, মেশিনের কাছে তত দূরে।

১ম প্রজন্ম: মেশিন ভাষা (Machine Language) নোট-১

মেশিন ভাষা

কম্পিউটারের মেশিনের নিজস্ব ভাষাকে মেশিন ভাষা বা **নিম্নস্তরের ভাষা** বলা হয়। সাধারণত মেশিন ভাষা 0 ও 1 এ দুই বাইনারি অঙ্ক দিয়ে লিখতে হয়।

✔ **সুবিধা**

- সবচেয়ে বড় সুবিধা: কম্পিউটারের সাথে **সরাসরি যোগাযোগ** করা যায়।
- প্রোগ্রাম নির্বাহের জন্য অনুবাদের প্রয়োজন হয় না; তাই কোনো অনুবাদক প্রোগ্রাম লাগে না, ফলে **দ্রুত নির্বাহ** হয়।
- প্রোগ্রাম রান করতে **অল্প মেমোরি** প্রয়োজন হয়।

✘ **অসুবিধা**

- শুধু 0 ও 1 ব্যবহার করা হয় বলে শেখা কষ্টকর ও প্রোগ্রাম লেখাও কষ্টসাধ্য।
- লেখা প্রোগ্রাম সাধারণত মানুষের জন্য বোঝা কঠিন।
- প্রোগ্রাম লিখতে প্রচুর সময় লাগে এবং ভুল হবার সম্ভাবনা খুব বেশি। ভুল হলে তা সনাক্ত করা ও দূর করা অর্থাৎ **ডিবাগিং কষ্টসাধ্য**।
- সবচেয়ে বড় অসুবিধা: এক ধরনের কম্পিউটারের জন্য লেখা প্রোগ্রাম অন্য ধরনের কম্পিউটারে চলে না — অর্থাৎ **যন্ত্র নির্ভর ভাষা**।

২য় প্রজন্ম: অ্যাসেম্বলি ভাষা (Assembly Language) নোট-১

অ্যাসেম্বলি ভাষা

যে ভাষায় বিভিন্ন সংকেত বা **নেমোনিক (Mnemonic)** ব্যবহার করে প্রোগ্রাম লেখা হয় তাকে অ্যাসেম্বলি ভাষা বলে। এতে 0 ও 1 ব্যবহার না করে বিভিন্ন সংকেত ব্যবহার করা হয়। উদাহরণ: `add eax, ebx` (ebx-এর মান eax-এর সাথে যোগ কর)।

অ্যাসেম্বলি ভাষায় প্রতিটি নির্দেশের ৪টি অংশ	
১. লেবেল (Label)	প্রোগ্রামের একটি নির্দিষ্ট স্থানকে চিহ্নিত করতে লেবেল ব্যবহৃত হয়।
২. অপ-কোড (Op-code)	অপ-কোডকে **অপারেশন কোড**-ও বলা হয়। অপ-কোডে নির্দেশ নেমোনিক থাকে (যেমন ADD, SUB, MOV)।
৩. অপারেন্ড (Operand)	অপকোড যার উপর কাজ করে তাকে অপারেন্ড বলে। এটি সাধারণত মেমোরির অ্যাড্রেস বা সরাসরি ডেটা হতে পারে।
৪. কমেন্ট (Comment)	প্রোগ্রামে কোনো নির্দেশের কী কাজ তা বর্ণনা আকারে লেখা হয়, যাকে কমেন্ট বা মন্তব্য বলে।

```
START: ADD EAX, EBX ; দুইটি মান যোগ
লেবেল  অপ-কোড অপারেন্ড  কমেন্ট
```

উদাহরণ চিত্র সংযোজন

🧠 **৪ অংশ: "লে-অপ-অপা-ক"**

লেবেল → **অপ**-কোড → **অপা**রেন্ড → **ক**মেন্ট (বাম থেকে ডানে এই ক্রমেই লেখা থাকে)।

✔ সুবিধা

- সহজে বোঝা যায় এবং প্রোগ্রাম রচনা যান্ত্রিক ভাষার তুলনায় অনেক সহজ।
- প্রোগ্রাম রচনা করতে সময় এবং শ্রম কম লাগে।
- প্রোগ্রাম সহজে পরিবর্তন করা যায়।
- প্রোগ্রামের ত্রুটি (Error) বের করে সমাধান করা অর্থাৎ ডিবাগিং মেশিন ভাষার চেয়ে সহজ।

✘ অসুবিধা

- সবচেয়ে বড় অসুবিধা: এক ধরনের কম্পিউটারের জন্য লেখা প্রোগ্রাম অন্য ধরনের কম্পিউটারে চলে না — **যন্ত্র নির্ভর ভাষা**।
- প্রোগ্রাম রচনার সময় প্রোগ্রামারকে মেশিন সম্পর্কে ধারণা থাকতে হয়।
- প্রোগ্রাম নির্বাহের জন্য অনুবাদক প্রোগ্রামের (**অ্যাসেম্বলার**) প্রয়োজন হয়।

৩য় প্রজন্ম: উচ্চস্তরের ভাষা (High Level Language) নোট-১

উচ্চস্তরের ভাষা

উচ্চস্তরের ভাষা হলো সেই সকল ভাষা যা **মানুষের বোধগম্য** এবং মানুষের ভাষার কাছাকাছি।

- ইংরেজি ভাষার সাথে মিল আছে।
- এই প্রোগ্রামিং ভাষা **যন্ত্র নির্ভর নয়**।
- এটি মানুষের জন্য বোঝা খুব সহজ কিন্তু computer সরাসরি বুঝতে পারে না।
- তৃতীয় প্রজন্মের ভাষা। যেমন: Fortran, Basic, Pascal, Cobol, C, C++, Visual Basic, Java ইত্যাদি।

⚠ সংশোধন

উৎসে: নোট-১ (পৃ. ৮, ** চিহ্নিত): "এই প্রোগ্রামিং ভাষা যন্ত্র নির্ভর"

সঠিক: উচ্চস্তরের ভাষা **যন্ত্র নির্ভর নয়** (machine independent)।

কেন: নোট-১-এরই পরের পৃষ্ঠার সুবিধার তালিকায় লেখা আছে: "এক মডেলের কম্পিউটারের জন্য লেখা প্রোগ্রাম অন্য মডেলে ব্যবহার করা যায় অর্থাৎ যন্ত্র নির্ভর নয়"। যন্ত্র নির্ভর হলো মেশিন ও অ্যাসেম্বলি ভাষা।

✔ সুবিধা

- উচ্চস্তরের ভাষা **প্রোগ্রামার বান্ধব**। তাই শেখা সহজ, প্রোগ্রাম লেখা সহজ ও লিখতে সময় কম লাগে।
- ভুল হবার সম্ভাবনা কম এবং ত্রুটি (Error) বের করে সংশোধন অর্থাৎ ডিবাগিং সহজ।
- প্রোগ্রাম লেখার জন্য কম্পিউটারের অভ্যন্তরীণ সংগঠন সম্পর্কে ধারণা থাকার প্রয়োজন নেই।
- এক মডেলের কম্পিউটারের জন্য লেখা প্রোগ্রাম অন্য মডেলে ব্যবহার করা যায় — **যন্ত্র নির্ভর নয়**।

✘ অসুবিধা

- কম্পিউটারের সাথে সরাসরি যোগাযোগ করা যায় না।
- প্রোগ্রামকে অনুবাদ করে কম্পিউটারকে বুঝিয়ে দিতে হয় — অর্থাৎ **অনুবাদক প্রোগ্রামের** (কম্পাইলার/ইন্টারপ্রিটার) প্রয়োজন হয়।
- নিম্নস্তরের প্রোগ্রামের চেয়ে তুলনামূলকভাবে **ধীর**।
- প্রোগ্রাম রান করতে **বেশি মেমোরি** প্রয়োজন হয়।

৪র্থ প্রজন্ম: 4GL নোট-১

- 4GL-এর পূর্ণরূপ **Fourth Generation Language**।
- ডেটাবেজ প্রোগ্রামিং এবং স্ক্রিপ্টিং-এর ক্ষেত্রে এই প্রজন্মের ভাষা ব্যবহৃত হয়।
- চতুর্থ প্রজন্মের ভাষা হলো **ডেটাবেজ সংক্রান্ত ভাষা**। (যেমন: SQL, Oracle Reports, Python-এর মতো স্ক্রিপ্টিং ভাষা) উদাহরণ সংযোজন

৫ম প্রজন্ম: 5GL নোট-১

- 5GL-এর পূর্ণরূপ **Fifth Generation Language**।
- **Artificial Intelligence** বা কৃত্রিম বুদ্ধিমত্তা নির্ভর যন্ত্র তৈরিতে বা গবেষণায় এই প্রজন্মের ভাষা ব্যবহৃত হয়।
- উদাহরণ: **PROLOG** (PROgramming LOGic), **LISP**, **Mercury** ইত্যাদি।

৩ স্তরের ভাষার তুলনা — এক নজরে সংযোজন

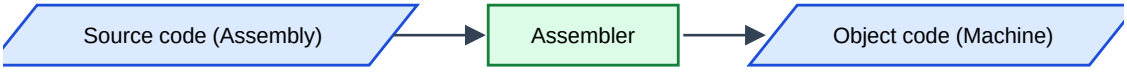
বৈশিষ্ট্য	মেশিন ভাষা	অ্যাসেম্বলি ভাষা	উচ্চস্তরের ভাষা
লেখা হয়	0 ও 1	নেমোনিক/সংকেত	ইংরেজির মতো শব্দ
অনুবাদক লাগে?	না	হ্যাঁ — অ্যাসেম্বলার	হ্যাঁ — কম্পাইলার/ইন্টারপ্রিটার
যন্ত্র নির্ভর?	হ্যাঁ	হ্যাঁ	না
নির্বাহ গতি	সবচেয়ে দ্রুত	দ্রুত	তুলনামূলক ধীর
মেমোরি	সবচেয়ে কম	কম	বেশি
লেখা/ডিবাগিং	সবচেয়ে কঠিন	মাঝারি	সবচেয়ে সহজ

৪ অনুবাদক প্রোগ্রাম

অ্যাসেম্বলার, কম্পাইলার, ইন্টারপ্রিটার

অনুবাদক প্রোগ্রাম নোট-১

যে প্রোগ্রাম অন্যসব ভাষাকে মেশিন ভাষায় রূপান্তর করে তাকে অনুবাদক প্রোগ্রাম বলে। অনুবাদক প্রোগ্রাম **৩ প্রকার**।



১. অ্যাসেম্বলার (Assembler)

অ্যাসেম্বলি ভাষায় লিখিত প্রোগ্রামকে মেশিন ভাষায় অনুবাদ করার জন্য অ্যাসেম্বলার ব্যবহার করা হয়। এটি অ্যাসেম্বলি ভাষায় লিখিত প্রোগ্রামকে কোড বা যান্ত্রিক ভাষায় রূপান্তর করে অর্থাৎ **নেমোনিক কোডকে মেশিন ভাষায়** অনুবাদ করে। প্রোগ্রামে কোনো ভুল থাকলে Error message দেয়।

২. কম্পাইলার (Compiler)

কম্পাইলার এক ধরনের অনুবাদক প্রোগ্রাম। কম্পাইলার হাই লেভেল ভাষায় লিখিত উৎস প্রোগ্রামকে বস্তু প্রোগ্রাম বা মেশিন ভাষার প্রোগ্রামে অনুবাদ করে। এটি **সেকেন্ডারি মেমোরিতে** থাকে। কোনো নির্দিষ্ট কম্পাইলার **একটিমাত্র** হাই লেভেল ভাষাকে মেশিন ভাষায় অনুবাদ করতে পারে (C-এর কম্পাইলার দিয়ে Java চলবে না)।

✔ কম্পাইলারের সুবিধা

- পুরো প্রোগ্রামটিকে **একেবারেই** বস্তু প্রোগ্রামে অনুবাদ করে।
- প্রোগ্রামে কোনো ভুল থাকলে তা জানিয়ে দেয়।
- প্রোগ্রামকে লিংক করে থাকে।
- প্রধান মেমোরিতে প্রয়োজনীয় মেমোরির ব্যবস্থা করে থাকে।

✘ কম্পাইলারের অসুবিধা

- ধাপে ধাপে ভুল সনাক্ত করা যায় না, ফলে সাথে সাথে সংশোধনও করা যায় না।
- মেমোরিতে বেশি জায়গার প্রয়োজন হয়।

৩. ইন্টারপ্রিটার (Interpreter)

ইন্টারপ্রিটার একটি অনুবাদক প্রোগ্রাম যা হাই লেভেল ভাষাকে মেশিন ভাষায় অনুবাদ করে। তবে কম্পাইলার যেখানে সম্পূর্ণ প্রোগ্রামকে অনুবাদ করে তারপর কার্যে পরিণত করে, সেখানে ইন্টারপ্রিটার **একটি লাইনের অনুবাদ করে তা কার্যে পরিণত করে, তারপর পরবর্তী নির্দেশে হাত দেয়**।

✔ ইন্টারপ্রিটারের সুবিধা

- এক লাইন এক লাইন করে কাজ করে।
- প্রোগ্রামের ভুল সংশোধন বা প্রোগ্রাম পরিবর্তন করা সহজ।
- এটি আকারে ছোট।
- অনূদিত প্রোগ্রামকে মেমোরিতে সেভ করে রাখতে হয় না।

✘ ইন্টারপ্রিটারের অসুবিধা

- প্রোগ্রামকে কাজের উপযোগী করতে অনেক সময় লাগে, কারণ এটি এক লাইন করে পড়ে ও অনুবাদ করে।
- রূপান্তরিত প্রোগ্রাম আবার চালাতে পুনরায় রূপান্তরের প্রয়োজন হয়।

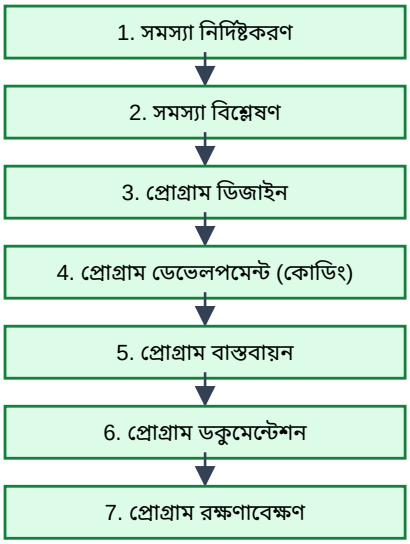
অ্যাসেম্বলার বনাম কম্পাইলার বনাম ইন্টারপ্রিটার *** নোট-১

বিষয়বস্তু	অ্যাসেম্বলার	কম্পাইলার ***	ইন্টারপ্রিটার ***
১. ধরন	অনুবাদক প্রোগ্রাম	অনুবাদক প্রোগ্রাম	অনুবাদক প্রোগ্রাম

২. সংজ্ঞা	যে অনুবাদক প্রোগ্রাম অ্যাসেম্বলি ভাষাকে মেশিন ভাষায় রূপান্তর করে	যে অনুবাদক প্রোগ্রাম সম্পূর্ণ প্রোগ্রামকে (একসাথে) মেশিন ভাষায় রূপান্তর করে	যে অনুবাদক প্রোগ্রাম এক লাইন করে মেশিন ভাষায় রূপান্তর করে
৩. ডিবাগিং	দ্রুত গতিসম্পন্ন	ধীর গতিসম্পন্ন	দ্রুত গতিসম্পন্ন
৪. প্রোগ্রামের আকার	ছোট	বড়	ছোট
৫. কার্যপদ্ধতি	সহজ	জটিল	সহজ
৬. মেমোরি	কম প্রয়োজন	বেশি প্রয়োজন	কম প্রয়োজন
৭. অনুবাদের পর চালানোর গতি সংযোজন	দ্রুত	**দ্রুত** (একবার অনুবাদ, বারবার চালানো)	**ধীর** (প্রতিবার অনুবাদ)
৮. উদাহরণ ভাষা সংযোজন	Assembly	C, C++	Python, BASIC

⚠ সংশোধন
উৎসে: নোট-১-এর টেবিলে অ্যাসেম্বলারের কার্যপদ্ধতি "ছোট"
সঠিক: কার্যপদ্ধতি **সহজ**
কেন: কার্যপদ্ধতির ক্ষেত্রে ছোট/বড় নয়, সহজ/জটিল লেখা হয়।

🧠 মনে রাখো: কম্পাইলার = "পুরো বই অনুবাদ", ইন্টারপ্রিটার = "দোভাষী"
কম্পাইলার পুরো বই একবারে অনুবাদ করে দেয় — সময় লাগে, জায়গা বেশি, কিন্তু পরে পড়া দ্রুত। ইন্টারপ্রিটার দোভাষীর মতো লাইনে লাইনে বলে — ভুল সাথে সাথে ধরা যায়, কিন্তু প্রতিবার আবার বলতে হয়।
কম্পাইলার একাই "বড়-বেশি-ধীর-জটিল"; অ্যাসেম্বলার ও ইন্টারপ্রিটার দুজনই "ছোট-কম-দ্রুত-সহজ"।



ধাপ নোট-১	কী হয়
১. সমস্যা নির্দিষ্টকরণ	যে সমস্যা সমাধানের জন্য প্রোগ্রাম রচনা করা হবে, তার একটি পরিষ্কার বর্ণনা তৈরি করা হয়। সমস্যাটির **ইনপুট ও আউটপুট** কী হবে ইত্যাদি এ ধাপে নির্ধারণ করা হয়।
২. সমস্যা বিশ্লেষণ	সমস্যা সমাধানের জন্য কী করণীয় তা পর্যালোচনা করে বিশ্লেষণ করা হয়। একাধিক সমাধান থাকলে ব্যবহারকারীর জন্য কোনটি বেশি উপযুক্ত তা দেখতে হয়।
৩. প্রোগ্রাম ডিজাইন	**অ্যালগরিদম ও ফ্লোচার্টের** সাহায্যে প্রোগ্রামের পূর্ণাঙ্গ পরিকল্পনা করা। প্রোগ্রাম উন্নয়নের **সবচেয়ে গুরুত্বপূর্ণ ধাপ**।
৪. ডেভেলপমেন্ট / কোডিং	কম্পিউটারের বোধগম্য ভাষায় প্রোগ্রাম রচনাকে কোডিং বলে। ফ্লোচার্ট ও অন্যান্য বিষয় বিবেচনা করে কম্পিউটারের ভাষায় নির্দেশসমূহ সাজিয়ে প্রোগ্রাম লেখা হয়।
৫. বাস্তবায়ন	প্রোগ্রাম রচনার পর সম্পূর্ণ প্রোগ্রাম পরীক্ষা করে দেখা হয়; প্রয়োজনীয় সংশোধনের মাধ্যমে প্রোগ্রামকে সম্পূর্ণভাবে তৈরি করা হয়।
৬. ডকুমেন্টেশন	ভুল সংশোধনের পর প্রোগ্রাম সঠিকভাবে কাজ করলে তাকে **Run Program** বলে। ভবিষ্যতে রক্ষণের জন্য এটি লিপিবদ্ধ করাকে প্রোগ্রাম লেখা বা **ডকুমেন্টেশন** বলে। এটি থাকলে পরবর্তী আপডেটেড ভার্সন তৈরি সহজ হয়।
৭. রক্ষণাবেক্ষণ	বিভিন্ন প্রয়োজনে ও উন্নতিকল্পে প্রোগ্রামের আধুনিকীকরণ, পরিবর্তন, পরিবর্ধন, ভুল সংশোধন ইত্যাদি।

🧠 **৭ ধাপ: "নি-বি-ডি-ডে-বা-ড-র"**

নির্দিষ্টকরণ → **বি**শ্লেষণ → **ডি**জাইন → **ডে**ভেলপমেন্ট → **বা**স্তবায়ন → **ড**কুমেন্টেশন → **র**ক্ষণাবেক্ষণ

বাক্য: "নিজের বিয়েতে ডিজে ডেকে বাজনা বাজাতে ডলার লাগে, রক্ষা নেই"।

একটি আদর্শ প্রোগ্রামের বৈশিষ্ট্য নোট-১

- সহজ, সংক্ষিপ্ত ও বোধগম্য
- কম মেমোরি লাগবে
- কম সময় লাগবে
- সহজে সম্প্রসারণযোগ্য
- Bug খুঁজে বের করা এবং ডিবাগ করা সহজ

Bug ও Debugging সংযোজন

প্রোগ্রামের ভুলকে **Bug** বলে, আর ভুল খুঁজে বের করে সংশোধন করাকে **Debugging** বলে। ভুল ৩ ধরনের: **Syntax error** (ব্যাকরণের ভুল, যেমন ; না দেওয়া — কম্পাইলার ধরে), **Logical error** (যুক্তির ভুল — প্রোগ্রাম চলে কিন্তু উত্তর ভুল), **Runtime error** (চলার সময় ভুল, যেমন 0 দিয়ে ভাগ)।

৬ অ্যালগরিদম (Algorithm)

লিখিত পরিকল্পনা

Algorithm নোট-২

কোনো সমস্যা সমাধানের জন্য ধারাবাহিক কতগুলো Command-এর **লিখিত রূপকে** Algorithm বলে।

- Algorithm একটি **বর্ণনামূলক** Program।
- এর দ্বারা সহজে Program-এর উদ্দেশ্য বুঝা যায়।
- তবে এর দ্বারা পুরো Program বুঝা কঠিন, এবং Algorithm-এর মাধ্যমে **প্রবাহের দিক বোঝা যায় না**।

Algorithm রচনার কতিপয় শর্ত নোট-২

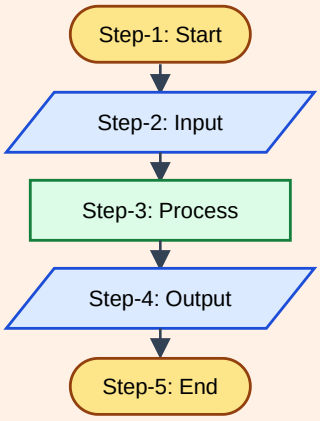
1. প্রথমে Program-এর বিষয় নির্ধারণ করতে হবে।
2. তারপর বিষয়টির শিরোনাম দিতে হবে।
3. সর্বশেষ বিষয়টিকে বিশ্লেষণ করে সিদ্ধান্তগুলোকে ধারাবাহিকভাবে সাজাতে হবে।

ভালো ও কার্যকর অ্যালগরিদমের বৈশিষ্ট্য নোট-১

1. অ্যালগরিদম সহজবোধ্য হবে।
2. প্রতিটি ধাপ স্পষ্টভাষ্য হবে — যেকোনো প্রোগ্রামার সহজেই বুঝতে পারবে।
3. সমস্যার সমাধান হবে **নির্দিষ্ট সংখ্যক ধাপে**।
4. এর ব্যাপকভাবে প্রয়োগ করা সম্ভব হবে।

অ্যালগরিদম সাধারণত **প্রোগ্রামিং ভাষার উপর নির্ভরশীল নয়**।

🎯 Algorithm-এর মৌলিক ধাপ ৫টি নোট-২



যেকোনো অ্যালগরিদম এই কাঠামোয় লিখলেই পূর্ণ নম্বর: **শুরু → ইনপুট → প্রক্রিয়া → আউটপুট → শেষ**।

৭ ফ্লোচার্ট (Flowchart) বা প্রবাহচিত্র

প্রতীক ও মৌলিক ছাঁচ

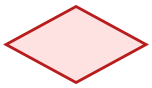
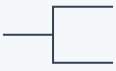
Flowchart নোট-১ + নোট-২

- যে চিত্রভিত্তিক পদ্ধতিতে বিশেষ কতকগুলো চিহ্নের সাহায্যে কোনো একটি নির্দিষ্ট সমস্যার সমাধান করা হয়, তাকে flowchart বলে। নোট-১
- কোনো সমস্যা সমাধানের জন্য ধারাবাহিক কতগুলো Command-এর চিত্রিত রূপকে Flow-chart বলে। নোট-২
- **অ্যালগরিদমের চিত্ররূপ (জ্যামিতিক রূপ) হলো ফ্লোচার্ট।**

ফ্লোচার্টের সুবিধা নোট-২

- Flow-chart একটি চিত্রভিত্তিক Program।
- Flow-chart দেখে Program বুঝতে সময় কম লাগে।
- এর দ্বারা প্রবাহের দিক নির্ণয় করা যায়।
- Program-এর ভুলত্রুটি খুঁজে বের করা ও সংশোধন করা সহজ।
- প্রতিটি ধাপ সহজেই বুঝা যায়।

ফ্লোচার্টের প্রতীকসমূহ

প্রতীক	নাম	কাজ	উৎস
	**টার্মিনাল / উপবৃত্ত (ডিম্বাকার)**	Program-এর **শুরু/শেষ** বুঝায় (Start / End)	নোট-২
	**সামান্তরিক**	**Input এবং Output** নির্দেশ করে (Input, Print)	নোট-২
	**আয়তক্ষেত্র**	**প্রক্রিয়াকরণের** জন্য (হিসাব: Sum = a+b)	নোট-২
	**হীরক (রম্বস)**	**শর্ত/সিদ্ধান্ত** গ্রহণে (Yes/No)	নোট-২
	**বৃত্ত (Circle) — সংযোগ**	সংযোগ প্রতীক। বড় flowchart এক পৃষ্ঠায় না ধরলে এ প্রতীক ব্যবহার করে বাকি অংশ অপর পৃষ্ঠায় আঁকা হয়; একাধিক শাখা মেলাতেও ব্যবহৃত।	নোট-১ নোট-২
	তীর চিহ্ন — প্রবাহদিক	প্রোগ্রামের ব্যবহার পথ নির্দেশ করে; একটি অপারেশনের পর পরবর্তীতে কোথায় যেতে হবে তা দেখায়	নোট-১ নোট-২
	**ষড়ভুজ — লুপ/চক্র**	কোনো কাজ বারবার করার জন্য (প্রস্তুতি/loop)	নোট-২
	**সাবরুটিন (দুই পাশে দাগযুক্ত আয়তক্ষেত্র)**	মূল প্রোগ্রামের কোনো অংশ বা সাবরুটিনকে চিত্রিত করা যায়; একটি প্রোগ্রামে একাধিক সাবরুটিন থাকতে পারে	নোট-১
	**বর্ণনা / টীকা**	কার্যক্রমের বর্ণনার জন্য; এটি **টীকা প্রতীক** নামেও পরিচিত	নোট-১

🧠 **প্রতীক মনে রাখার কৌশল**

ডিম দিয়ে শুরু-শেষ (জীবনও ডিম থেকে শুরু) • **হেলানো** বাক্স = ইনপুট/আউটপুট (জিনিস ঢোকে-বের হয়) • **সোজা** বাক্স = কাজ • **হীরা** = সিদ্ধান্ত (হীরা কিনব কি কিনব না?) • **গোল** = জোড়া লাগানো।

Flowchart গঠনের ৫টি মৌলিক ছাঁচ (Logic Structure) নোট-১

ছাঁচ	English	কখন	উদাহরণ (এই নোটে)
১. সরল অনুক্রম	Linear Sequential logic	ধাপগুলো একটির পর একটি, কোনো শর্ত নেই	ক্ষেত্রফল, যোগফল, C→F
২. সিদ্ধান্ত বা নির্বাচন	Decision logic	শর্ত অনুযায়ী দুই পথের একটি	বৃহত্তম/ক্ষুদ্রতম, ধনাত্মক-ঋণাত্মক
৩. পুনরাবৃত্তি বা রিপিটেশন	Looping logic	একই কাজ বারবার	Bangladesh ১২ বার, ১–১০০
৪. জাম্প বা শাখা	Jump or Branching	সরাসরি অন্য ধাপে চলে যাওয়া	"৩ নং ধাপে যাও"
৫. সাবরুটিন	Subroutine or Function logic	আলাদা ছোট প্রোগ্রাম ডেকে আনা	sqrt(), pow() ফাংশন

অ্যালগরিদম বনাম ফ্লোচার্ট নোট-২ + সংযোজন

অ্যালগরিদম	ফ্লোচার্ট
সমাধানের **লিখিত** (বর্ণনামূলক) রূপ	সমাধানের **চিত্রিত** (জ্যামিতিক) রূপ
প্রবাহের দিক বোঝা যায় না	তীর দিয়ে প্রবাহের দিক বোঝা যায়
বড় সমস্যায় বুঝতে সময় বেশি লাগে	দেখে বুঝতে সময় কম লাগে
আগে লেখা হয়	অ্যালগরিদমের পরে আঁকা হয়
কোনো প্রতীক লাগে না	নির্দিষ্ট প্রতীক ব্যবহার করতে হয়

৮ সমাধানকৃত অ্যালগরিদম ও ফ্লোচার্ট

নোট-১-এর ১২টি সমস্যা + C কোড

⚡ যেকোনো A-F-C লেখার সূত্র

১. ইনপুট কী, আউটপুট কী — আগে ঠিক করো। **২.** সূত্রটা C ভাষায় লেখো (× → *, ÷ → /)। **৩.** অ্যালগরিদম = শুরু, ইনপুট, প্রক্রিয়া, আউটপুট, শেষ। **৪.** ফ্লোচার্ট = প্রতিটি ধাপের জন্য সঠিক প্রতীক। **৫.** শর্ত থাকলে হীরা, পুনরাবৃত্তি থাকলে ফিরতি তীর। নিচের প্রতিটি C কোড আসলেই কম্পিউটারে চালিয়ে আউটপুট দেখানো হয়েছে।

৮.১. ত্রিভুজের ক্ষেত্রফল নির্ণয় নোট-১

Algorithm

ধাপ-1: শুরু।

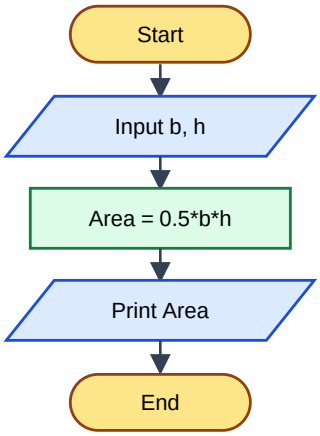
ধাপ-2: ত্রিভুজের ভূমি (b) ও উচ্চতা (h)-এর মান গ্রহণ করা।

ধাপ-3: ক্ষেত্রফল = ½ × ভূমি × উচ্চতা সূত্র ব্যবহার করে ক্ষেত্রফল নির্ণয় করা।

ধাপ-4: ত্রিভুজের ক্ষেত্রফল প্রদর্শন করা।

ধাপ-5: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    float b, h, area;
    scanf("%f %f", &b, &h);
    area = 0.5 * b * h;
    printf("Area = %.2f", area);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 10 6
Area = 30.00
```

৮.২. তিনটি ধনাত্মক পূর্ণসংখ্যার যোগফল নোট-১

Algorithm

ধাপ-1: শুরু।

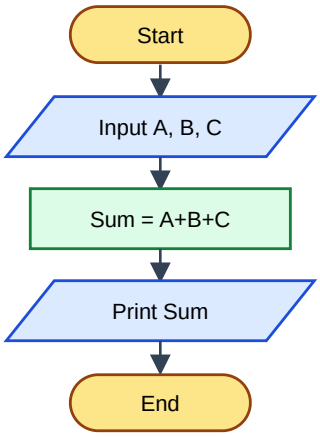
ধাপ-2: ইনপুট হিসেবে A, B, C চলকের মান গ্রহণ।

ধাপ-3: A, B, C চলকের মান যোগ করে Sum চলকে রাখা।

ধাপ-4: ফলাফল হিসেবে Sum চলকের মান প্রদর্শন করা।

ধাপ-5: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int A, B, C, Sum;
    scanf("%d %d %d", &A, &B, &C);
    Sum = A + B + C;
    printf("Sum = %d", Sum);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

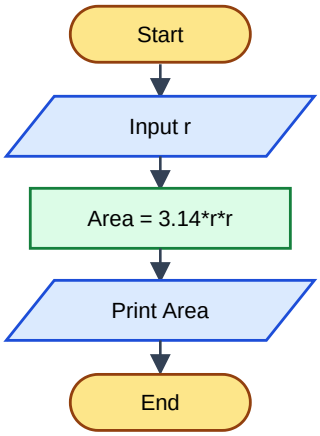
```
ইনপুট দিলাম: 4 9 12
Sum = 25
```

৮.৩. বৃত্তের ক্ষেত্রফল নির্ণয় নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে বৃত্তের ব্যাসার্ধ r-এর মান গ্রহণ। π-এর মান = 3.14 নির্দিষ্ট করি।
ধাপ-3: Area = π × r × r ব্যবহার করে Area-এর মান বের করি।
ধাপ-4: ফলাফল হিসেবে Area-এর মান প্রদর্শন।
ধাপ-5: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    float r, Area;
    scanf("%f", &r);
    Area = 3.14 * r * r;
    printf("Area = %.2f", Area);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

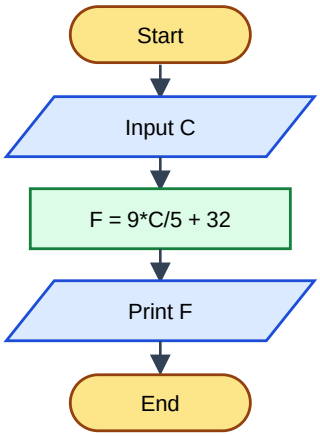
```
ইনপুট দিলাম: 5
Area = 78.50
```

৮.৪. সেলসিয়াসকে ফারেনহাইটে রূপান্তর নোট-১ + নোট-২

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে সেলসিয়াস তাপমাত্রা C-এর মান গ্রহণ।
ধাপ-3: F = 9 × C / 5 + 32 ব্যবহার করে F-এর মান নির্ণয়।
ধাপ-4: F-এর মান প্রদর্শন।
ধাপ-5: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    float c, f;
    scanf("%f", &c);
    f = ((9 * c) / 5) + 32;
    printf("F = %.2f", f);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

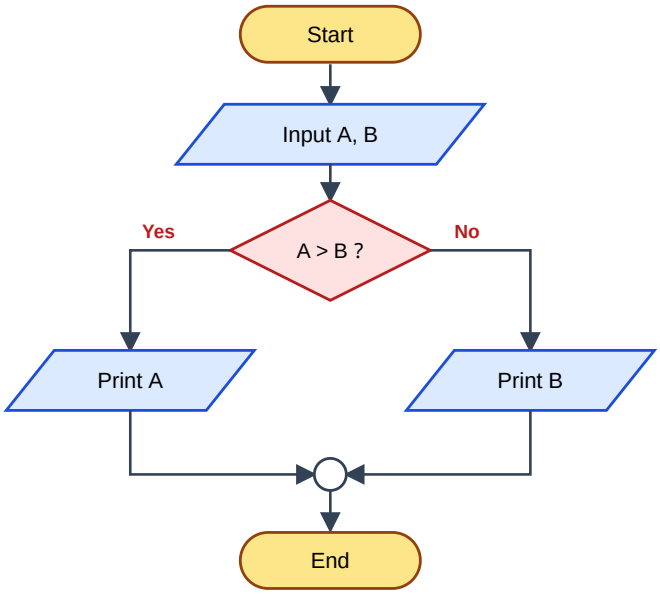
```
ইনপুট দিলাম: 100
F = 212.00
```

৮.৫. দুইটি সংখ্যার মধ্যে বৃহত্তম নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে A ও B-এর মান গ্রহণ।
ধাপ-3: A > B ?
ক) হ্যাঁ হলে, A প্রদর্শন করি।
খ) না হলে, B প্রদর্শন করি।
ধাপ-4: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int A, B;
    scanf("%d %d", &A, &B);
    if (A > B)
        printf("Largest = %d", A);
    else
        printf("Largest = %d", B);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

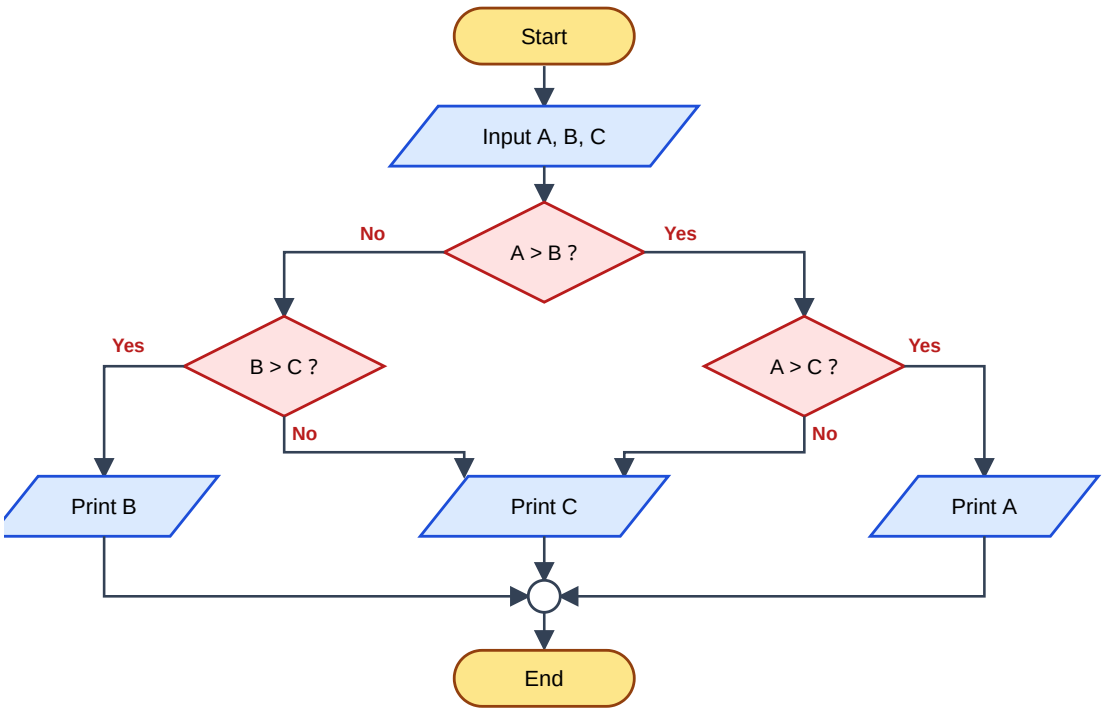
```
ইনপুট দিলাম: 15 22
Largest = 22
```

৮.৬. তিনটি সংখ্যার মধ্যে বৃহত্তম *** নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে তিনটি সংখ্যা A, B, C-এর মান গ্রহণ।
ধাপ-3: A > B কি সত্য?
ক) হ্যাঁ হলে, ৪ নং ধাপে যাও।
খ) না হলে, ৫ নং ধাপে যাও।
ধাপ-4: A > C কি সত্য?
ক) হ্যাঁ হলে, A প্রদর্শন করি, ৬ নং ধাপে যাও।
খ) না হলে, C প্রদর্শন করি, ৬ নং ধাপে যাও।
ধাপ-5: B > C কি সত্য?
ক) হ্যাঁ হলে, B প্রদর্শন করি।
খ) না হলে, C প্রদর্শন করি।
ধাপ-6: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int A, B, C;
    scanf("%d %d %d", &A, &B, &C);
    if (A > B)
    {
        if (A > C) printf("Largest = %d", A);
        else       printf("Largest = %d", C);
    }
    else
    {
        if (B > C) printf("Largest = %d", B);
        else       printf("Largest = %d", C);
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 12 45 30
Largest = 45
```

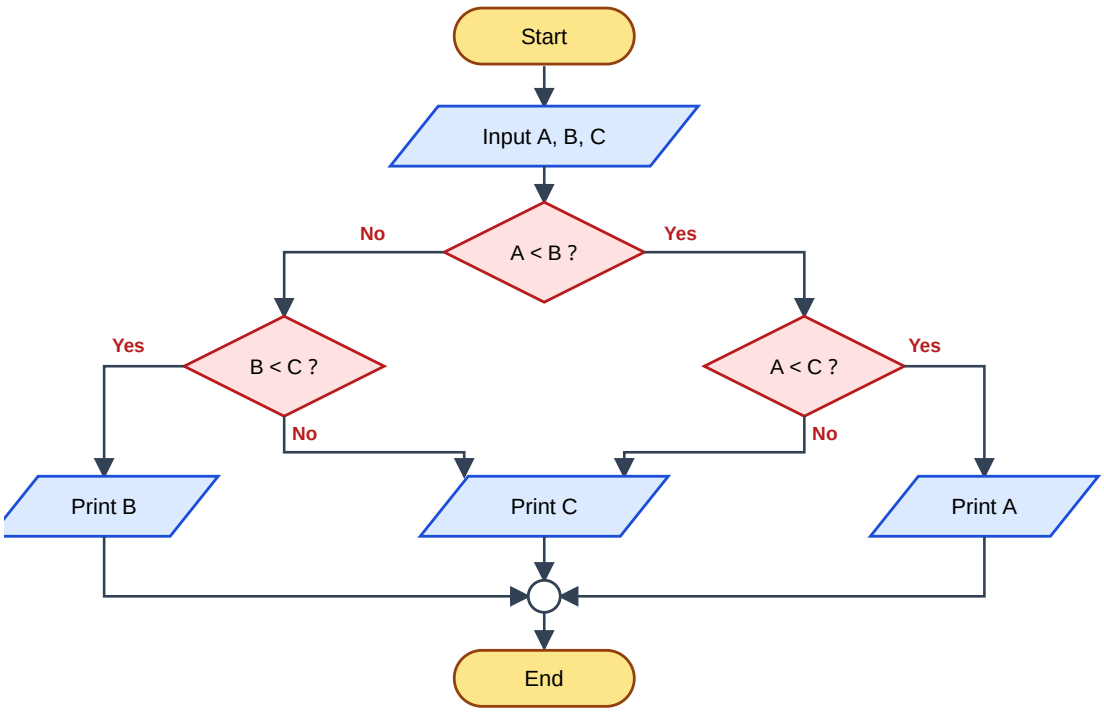
⚠ সংশোধন
উৎসে: নোট-১ (৮.৬): ধাপ-৪-এ A বা C প্রদর্শনের পর "৬ নং ধাপে যাও" লেখা নেই
সঠিক: ধাপ-৪-এর দুই শাখার শেষে "**৬ নং ধাপে যাও**" লিখতে হবে (ক্ষুদ্রতমের অ্যালগরিদমে যেমন আছে)।
কেন: না লিখলে ধাপ-৪ শেষে ধাপ-৫-এ চলে গিয়ে আবার B ও C তুলনা করবে — দুইবার উত্তর ছাপা হতে পারে।

৮.৭. তিনটি সংখ্যার মধ্যে ক্ষুদ্রতম নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে তিনটি সংখ্যা A, B, C-এর মান গ্রহণ।
ধাপ-3: A < B কি সত্য?
ক) হ্যাঁ হলে, ৪ নং ধাপে যাও।
খ) না হলে, ৫ নং ধাপে যাও।
ধাপ-4: A < C কি সত্য?
ক) হ্যাঁ হলে, A প্রদর্শন কর, ৬ নং ধাপে যাও।
খ) না হলে, C প্রদর্শন কর, ৬ নং ধাপে যাও।
ধাপ-5: B < C কি সত্য?
ক) হ্যাঁ হলে, B প্রদর্শন করি।
খ) না হলে, C প্রদর্শন করি।
ধাপ-6: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int A, B, C;
    scanf("%d %d %d", &A, &B, &C);
    if (A < B)
    {
        if (A < C) printf("Smallest = %d", A);
        else       printf("Smallest = %d", C);
    }
    else
    {
        if (B < C) printf("Smallest = %d", B);
        else       printf("Smallest = %d", C);
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 12 45 30
Smallest = 12
```

⚡ বৃহত্তম ↔ ক্ষুদ্রতম শর্টকাট

একটি মুখস্থ করলেই হবে: বৃহত্তমের অ্যালগরিদম/ফ্লোচার্ট/কোডে শুধু **> চিহ্নকে < করে দাও** — বাকি সব হুবহু এক।

৮.৮. সংখ্যাটি ধনাত্মক, ঋণাত্মক না শূন্য নোট-১

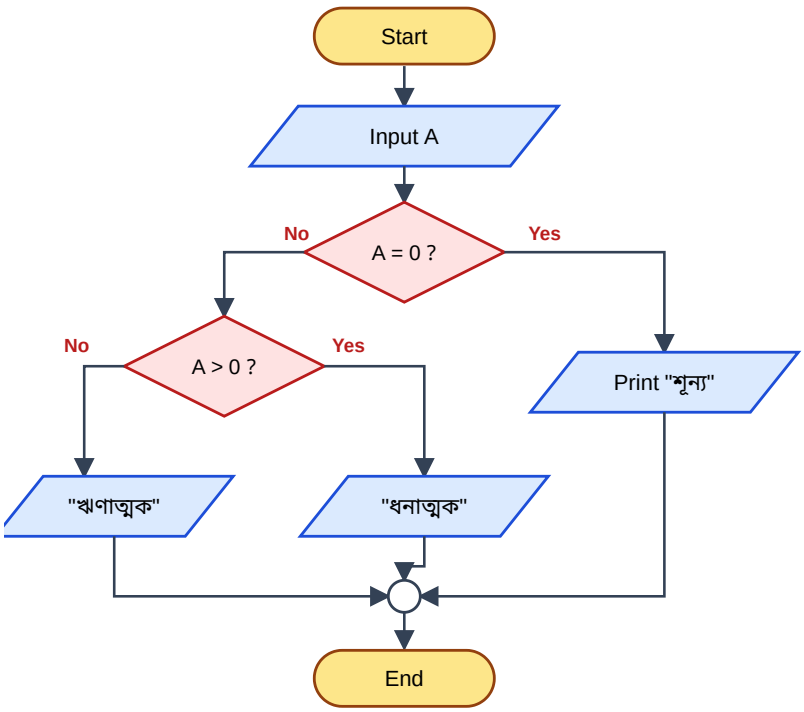
Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট হিসেবে A-এর মান গ্রহণ।
ধাপ-3: A = 0 ? হ্যাঁ হলে "শূন্য" প্রদর্শন করে ৬ নং ধাপে যাও।
ধাপ-4: A > 0 ? হ্যাঁ হলে "ধনাত্মক" প্রদর্শন করে ৬ নং ধাপে যাও।
ধাপ-5: "ঋণাত্মক" প্রদর্শন।
ধাপ-6: শেষ।

⚠

নোট-১-এ এই সমস্যার শুধু ফ্লোচার্ট ছিল; অ্যালগরিদমটি সংযোজন। এছাড়া ফ্লোচার্টে "A=0" ছাপার ঘরে আসলে "**শূন্য**" প্রদর্শন বোঝানো হয়েছে। C-তে তুলনার জন্য `==` লাগে, `=` নয়।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int A;
    scanf("%d", &A);
    if (A == 0)
        printf("Zero");
    else if (A > 0)
        printf("Positive");
    else
        printf("Negative");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

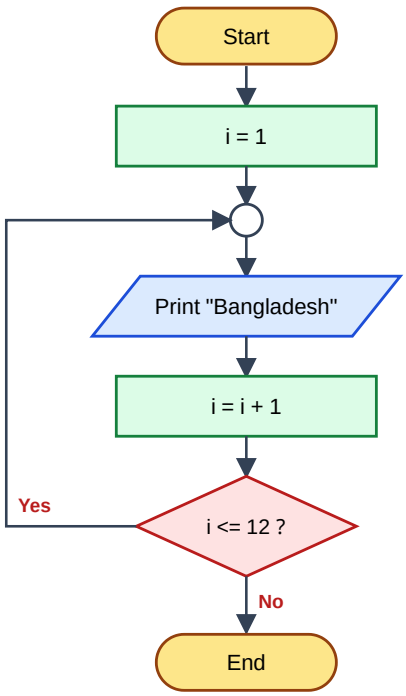
```
ইনপুট দিলাম: -7
Negative
```

৮.৯. "Bangladesh" লেখাটি ১২ বার প্রদর্শন নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: i চলকের মান 1 ধরি।
ধাপ-3: "Bangladesh" লেখাটি প্রদর্শন।
ধাপ-4: i-এর মান 1 করে বাড়াই।
ধাপ-5: i-এর মান 12-এর চেয়ে ছোট বা সমান হলে ৩ নং ধাপে যাই।
ধাপ-6: প্রোগ্রাম শেষ করি।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 12; i++)
        printf("Bangladesh\n");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

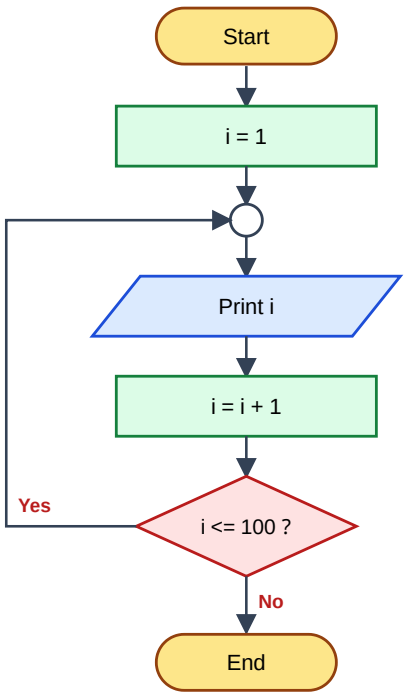
```
ইনপুট দিলাম: –
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
Bangladesh
```

৮.১০. ১ থেকে ১০০ পর্যন্ত সংখ্যাগুলো প্রদর্শন নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: i চলকের মান 1 ধরি।
ধাপ-3: ফলাফল হিসেবে i-এর মান প্রদর্শন করি।
ধাপ-4: i-এর মান 1 করে বাড়াই।
ধাপ-5: i-এর মান 100-এর চেয়ে ছোট **বা সমান** হলে ৩ নং ধাপে যাই।
ধাপ-6: প্রোগ্রাম শেষ করি।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 100; i++)
        printf("%d ", i);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

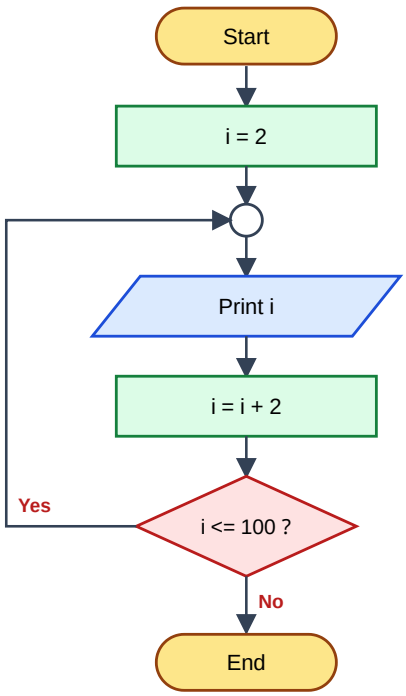
```
ইনপুট দিলাম: –
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17
18 19 20 21 22 23 24 25 26 27 28 29 30 31
32 33 34 35 36 37 38 39 40 41 42 43 44 45
46 47 48 49 50 51 52 53 54 55 56 57 58 59
60 61 62 63 64 65 66 67 68 69 70 71 72 73
74 75 76 77 78 79 80 81 82 83 84 85 86 87
88 89 90 91 92 93 94 95 96 97 98 99 100
```

৮.১১. ১ থেকে ১০০ পর্যন্ত জোড় সংখ্যাগুলো প্রদর্শন নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: i চলকের মান 2 ধরি।
ধাপ-3: ফলাফল হিসেবে i-এর মান প্রদর্শন করি।
ধাপ-4: i-এর মান 2 করে বাড়াই।
ধাপ-5: i-এর মান 100-এর চেয়ে ছোট **বা সমান** হলে ৩ নং ধাপে যাই।
ধাপ-6: প্রোগ্রাম শেষ করি।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int i;
    for (i = 2; i <= 100; i = i + 2)
        printf("%d ", i);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

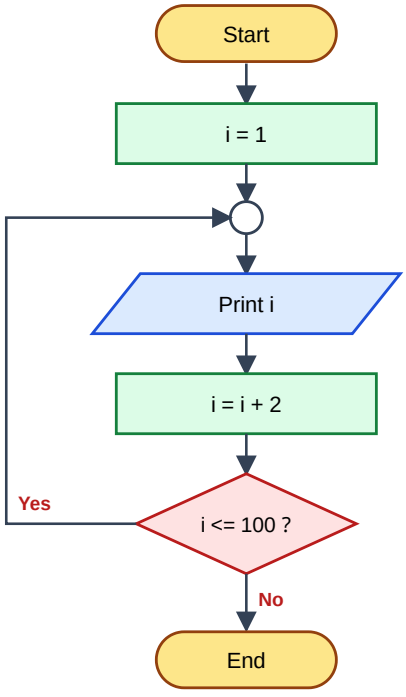
```
ইনপুট দিলাম: –
2 4 6 8 10 12 14 16 18 20 22 24 26 28 30
32 34 36 38 40 42 44 46 48 50 52 54 56 58
60 62 64 66 68 70 72 74 76 78 80 82 84 86
88 90 92 94 96 98 100
```

৮.১২. ১ থেকে ১০০ পর্যন্ত বিজোড় সংখ্যাগুলো প্রদর্শন নোট-১

Algorithm

ধাপ-1: শুরু।
ধাপ-2: i চলকের মান 1 ধরি।
ধাপ-3: ফলাফল হিসেবে i-এর মান প্রদর্শন করি।
ধাপ-4: i-এর মান 2 করে বাড়াই।
ধাপ-5: i-এর মান 100-এর চেয়ে ছোট (বা সমান) হলে ৩ নং ধাপে যাই।
ধাপ-6: প্রোগ্রাম শেষ করি।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int i;
    for (i = 1; i <= 100; i = i + 2)
        printf("%d ", i);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
1 3 5 7 9 11 13 15 17 19 21 23 25 27 29
31 33 35 37 39 41 43 45 47 49 51 53 55 57
59 61 63 65 67 69 71 73 75 77 79 81 83 85
87 89 91 93 95 97 99
```

⚠ সংশোধন

উৎসে: নোট-১ (জোড় সংখ্যা): "i-এর মান 100-এর চেয়ে ছোট হয়, তাহলে ৩ নং ধাপে যাই"

সঠিক: "100-এর চেয়ে **ছোট বা সমান** (i ≤ 100) হলে ৩ নং ধাপে যাই"

কেন: শুধু "ছোট" লিখলে i = 100 হলে লুপ থেমে যায়, ফলে **100 ছাপা হবে না**। নোটের ফ্লোচার্টে ঠিকই i ≤ 100 আছে — অ্যালগরিদম আর ফ্লোচার্ট মিলতে হবে। (বিজোড়ের ক্ষেত্রে ১০০ নিজে বিজোড় নয়, তাই সেখানে সমস্যা হয় না; তবুও ≤ লেখাই নিরাপদ।)

⚡ লুপের ৪টি জিনিস মুখস্থ — "শুরু, শর্ত, কাজ, বাড়াও"

প্রতিটি লুপ প্রশ্নে শুধু এই ৪টা বদলায়:

প্রশ্ন	শুরু	বাড়াও	শর্ত
১–১০০ সব	i=1	i+1	i≤100
জোড়	i=2	i+2	i≤100
বিজোড়	i=1	i+2	i≤100
"X" n বার	i=1	i+1	i≤n
১০০ থেকে ১ উল্টো সংযোজন	i=100	i−1	i≥1

৮ক আরও সমাধানকৃত অ্যালগরিদম, ফ্লোচার্ট ও C কোড

ফাইনাল নোট থেকে নতুন ১০টি সমস্যা + সিরিজ-সমষ্টি

⚡ একই প্যাটার্ন

সব সূত্রভিত্তিক সমস্যা: **Start → Input → সূত্র → Print → End**। শুধু সূত্রের লাইন বদলায়। শর্ত থাকলে মাঝে হীরা (◇), লুপ থাকলে ফিরতি তীর।

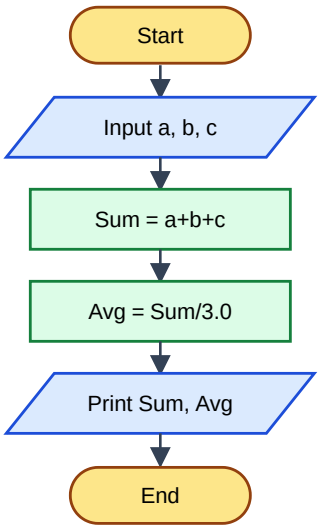
৮ক.১. দুই/তিনটি সংখ্যার যোগফল ও গড়

নোট-২ + ফাইনাল নোট

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট a, b, c গ্রহণ।
ধাপ-3: Sum = a + b + c
ধাপ-4: Avg = Sum / 3 (দুইটি সংখ্যা হলে /2)
ধাপ-5: Sum ও Avg প্রদর্শন।
ধাপ-6: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int a, b, c, sum;
    float avg;
    scanf("%d %d %d", &a, &b, &c);
    sum = a + b + c;
    avg = sum / 3.0;    /* 3 নয়, 3.0 */
    printf("Sum = %d, Avg = %.2f", sum, avg);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 4 5 7
Sum = 16, Avg = 5.33
```

৮ক.২. ফারেনহাইটকে সেলসিয়াসে রূপান্তর

নোট-২ + ফাইনাল নোট

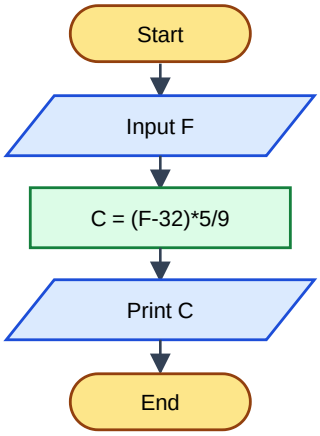
Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট F গ্রহণ।
ধাপ-3: C = (F − 32) × 5 / 9
ধাপ-4: C প্রদর্শন।
ধাপ-5: শেষ।

প্রতিপাদন

C/5 = (F−32)/9 ⇒ **C = 5(F−32)/9**

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    float f, c;
    scanf("%f", &f);
    c = (f - 32) * 5 / 9;
    printf("C = %.2f", c);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

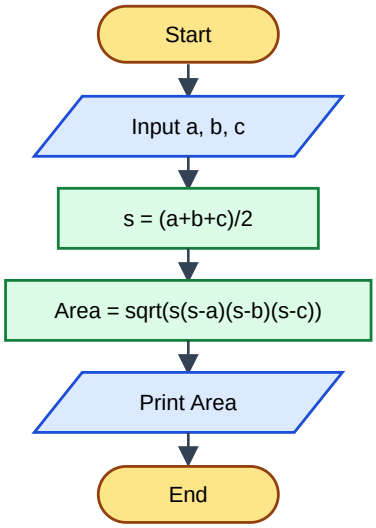
```
ইনপুট দিলাম: 212
C = 100.00
```

৮ক.৩. তিন বাহু দেওয়া ত্রিভুজের ক্ষেত্রফল (হেরনের সূত্র) নোট-২ + ফাইনাল নোট

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট a, b, c (তিন বাহু)।
ধাপ-3: s = (a + b + c) / 2
ধাপ-4: Area = √(s(s−a)(s−b)(s−c))
ধাপ-5: Area প্রদর্শন।
ধাপ-6: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
#include <math.h>
int main()
{
    float a, b, c, s, area;
    scanf("%f %f %f", &a, &b, &c);
    s = (a + b + c) / 2;
    area = sqrt(s*(s-a)*(s-b)*(s-c));
    printf("Area = %.2f", area);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

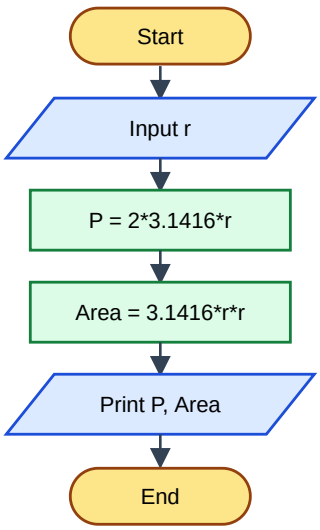
```
ইনপুট দিলাম: 3 4 5
Area = 6.00
```

৮ক.৪. বৃত্তের পরিধি ও ক্ষেত্রফল ফাইনাল নোট + হাতে-লেখা নোট

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট r (ব্যাসার্ধ)।
ধাপ-3: P = 2 × 3.1416 × r
ধাপ-4: Area = 3.1416 × r × r
ধাপ-5: P ও Area প্রদর্শন।
ধাপ-6: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    float radius, area, perimeter;
    printf("Enter radius: ");
    scanf("%f", &radius);
    perimeter = 2 * 3.1416 * radius;
    area = 3.1416 * radius * radius;
    printf("\nPerimeter: %f\n", perimeter);
    printf("The area of the circle is: %f", area);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 7
Enter radius:
Perimeter: 43.982399
The area of the circle is: 153.938400
```

৮ক.৫. সংখ্যাটি জোড় নাকি বিজোড় ফাইনাল নোট

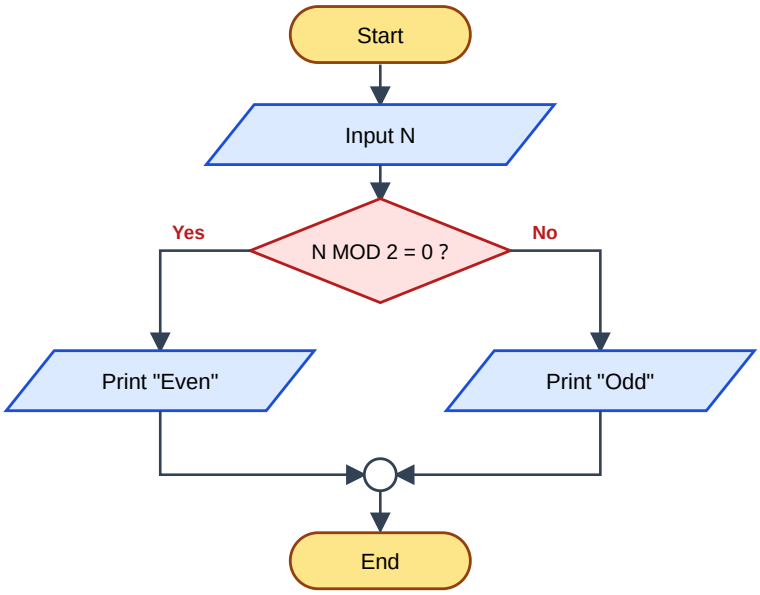
Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট N।
ধাপ-3: N MOD 2 = 0 ?
 হ্যাঁ → "Even Number" প্রদর্শন
 না → "Odd Number" প্রদর্শন
ধাপ-4: শেষ।

MOD = %

অ্যালগরিদমে **MOD**, C-তে **%** — দুটোই ভাগশেষ। 17 % 2 = 1 → বিজোড়।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int a;
    scanf("%d", &a);
    if (a % 2 == 0)
        printf("%d is even\n", a);
    else
        printf("%d is odd\n", a);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

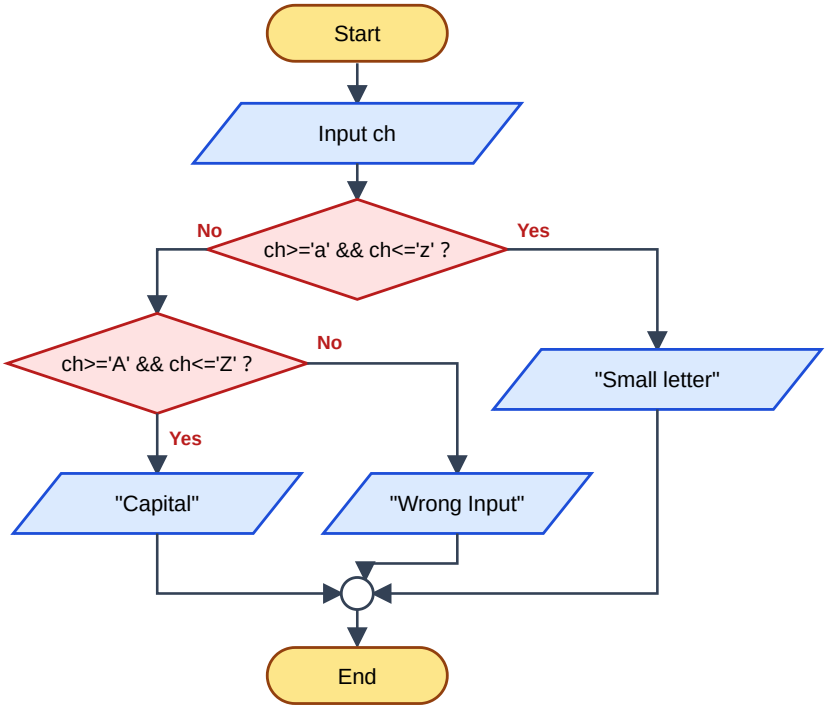
```
ইনপুট দিলাম: 17
17 is odd
```

৮ক.৬. অক্ষরটি ছোট হাতের নাকি বড় হাতের ফাইনাল নোট

Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট ch।
ধাপ-3: (ch ≥ 'a' AND ch ≤ 'z') ? হ্যাঁ → "Small letter"; না → ধাপ-৪
ধাপ-4: (ch ≥ 'A' AND ch ≤ 'Z') ? হ্যাঁ → "Capital letter"; না → "Wrong Input"
ধাপ-5: শেষ।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    char ch;
    scanf("%c", &ch);
    if (ch >= 'a' && ch <= 'z')
        printf("Small letter");
    else if (ch >= 'A' && ch <= 'Z')
        printf("Capital letter");
    else
        printf("Wrong Input");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: G
Capital letter
```

৮ক.৭. কোনো বছর Leap Year কিনা *** ফাইনাল নোট

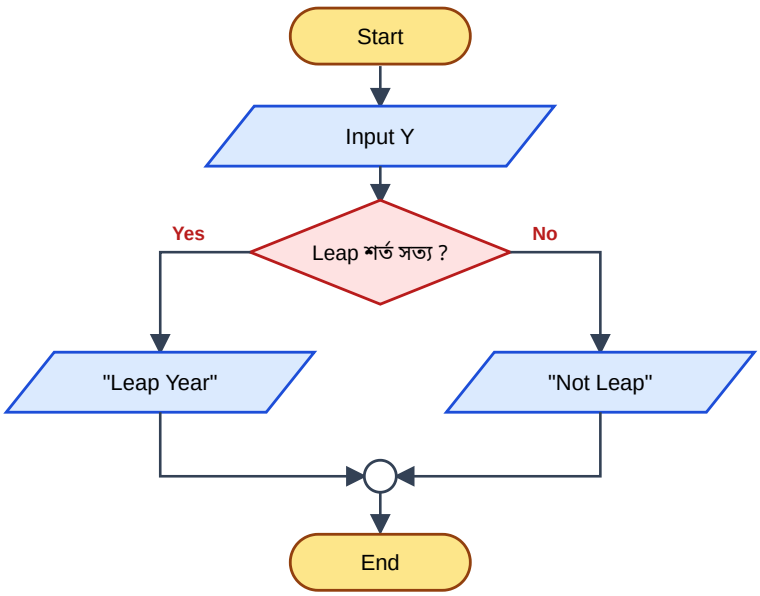
Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট Y।
ধাপ-3: (Y MOD 400 = 0) OR (Y MOD 4 = 0 AND Y MOD 100 ≠ 0) ?
হ্যাঁ → "Leap Year"
না → "Not a Leap Year"
ধাপ-4: শেষ।

🧠 **মনে রাখো**

"৪০০ হলে নিশ্চিত, ১০০ হলে বাদ, ৪ হলে হ্যাঁ"
2000 → leap (÷400), 1900 → নয় (÷100 কিন্তু ÷400 না), 2024 → leap।

Flowchart



Leap শর্ত = (Y%400==0) || (Y%4==0 && Y%100!=0)

C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int year;
    scanf("%d", &year);
    if (year % 400 == 0)
        printf("%d Leap year", year);
    else
    {
        if (year % 100 != 0 && year % 4 == 0)
            printf("%d Leap year", year);
        else
            printf("%d not leap year", year);
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 1900
1900 not leap year
```

৮ক.৮. দুইটি সংখ্যার গ.সা.গু (GCD — ইউক্লিডীয় পদ্ধতি) ফাইনাল নোট

Algorithm

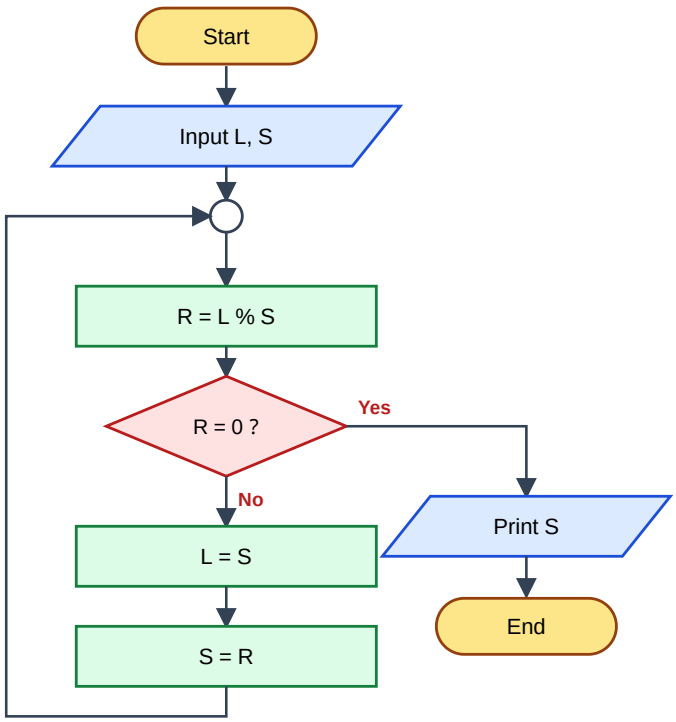
ধাপ-1: শুরু।
ধাপ-2: ইনপুট L, S (L > S)।
ধাপ-3: R = L MOD S
ধাপ-4: R = 0 ?
হ্যাঁ → S প্রদর্শন (এটিই গ.সা.গু), শেষ।
না → L = S, S = R; ধাপ-৩ এ যাও।

ট্রেস (L=30, S=4)

R=30%4=2 → L=4, S=2 → R=4%2=0 → থামে → গ.সা.গু = **2**।

C কোডে লুপের ভেতরে L=S আগে হয়ে যায় বলে শেষে **L** প্রিন্ট হয় — অ্যালগরিদমের S আর কোডের L একই মান।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int L, S, R;
    scanf("%d", &L);
    scanf("%d", &S);
    do
    {
        R = L % S;
        L = S;
        S = R;
    } while (R != 0);
    printf("GCD = %d", L);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 30 4
GCD = 2
```

৮ক.৯. একটি সংখ্যার Factorial (N!) ফাইনাল নোট

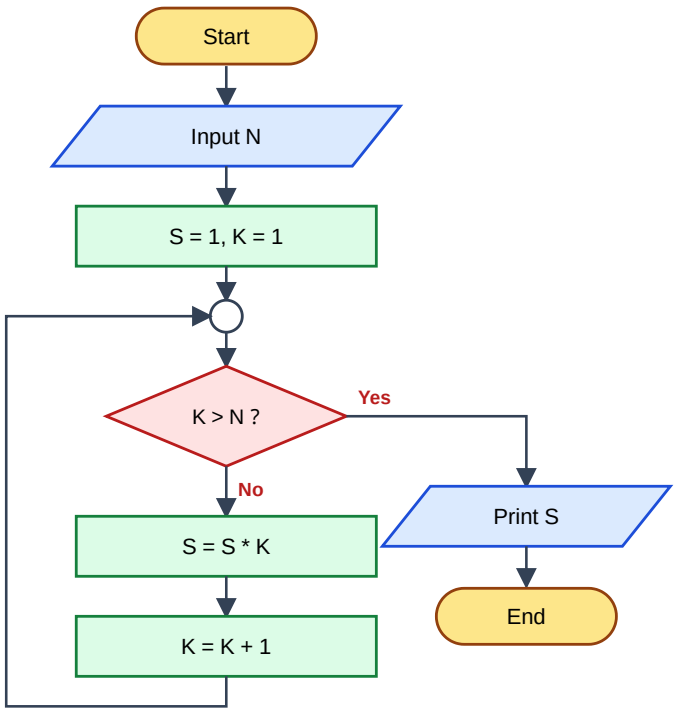
Algorithm

ধাপ-1: শুরু।
ধাপ-2: ইনপুট N।
ধাপ-3: S = 1, K = 1
ধাপ-4: K > N ?
হ্যাঁ → S প্রদর্শন, শেষ।
না → S = S × K, K = K + 1; ধাপ-৪ এ ফিরে যাও।

⚠ ফাঁদ

গুণফলের শুরুর মান **S=1**; ভুলে S=0 দিলে উত্তর সবসময় 0। 0! = 1।

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
int main()
{
    int N, K, S;
    scanf("%d", &N);
    S = 1;                  /* গুণফলে শুরু 1, যোগে 0 */
    for (K = 1; K <= N; K = K + 1)
        S = S * K;
    printf("%d! = %d", N, S);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 5
5! = 120
```

৮ক.১০ শুধু সংখ্যা প্রদর্শনের লুপ — পূর্ণ রেফারেন্স টেবিল ফাইনাল নোট

সমস্যা	শুরু	আপডেট	শর্ত	প্রিন্ট
১–১০০ সব সংখ্যা	i=1	i=i+1	i≤100	i
১–১০০ জোড়	i=2	i=i+2	i≤100	i
১–১০০ বিজোড়	i=1	i=i+2	i≤100	i
১–১০০ পর্যন্ত ৫-এর গুণিতক (নতুন)	i=5	i=i+5	i≤100	i
১–১০০ পর্যন্ত সংখ্যার বর্গ (নতুন)	i=1	i=i+1	i≤100	**i*i**
"Bangladesh" ১২ বার	i=1	i=i+1	i≤12	"Bangladesh"

৫-এর গুণিতক ও বর্গ (একই কাঠামো)

```c
#include <stdio.h>
int main()
{
    int i;
    printf("5-er gunitok: ");
    for (i = 5; i <= 100; i = i + 5)
        printf("%d ", i);
    printf("\nborgo (1-10): ");
    for (i = 1; i <= 10; i++)
        printf("%d ", i * i);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: —
5-er gunitok: 5 10 15 20 25 30 35 40 45
50 55 60 65 70 75 80 85 90 95 100
borgo (1-10): 1 4 9 16 25 36 49 64 81 100
```

৮খ সিরিজ-সমষ্টি (যোগফল) নির্ণয় — মাস্টার টেবিল

Goto-স্টাইল + ট্রেস টেবিল + চালানো C কোড

সঞ্চয়ী চলক (Accumulator)

যে চলকে প্রতিবার নতুন পদ যোগ/গুণ হয়ে জমতে থাকে (S)। যোগের জন্য শুরু **S=0**, গুণের জন্য **S=1**। আর K হলো পদ-গণক (counter)।

সিরিজ	শুরুর মান	লুপের ভেতরে	K-এর আপডেট	থামার শর্ত
1+2+3+…+N	S=0, K=1	S=S+K	K=K+1	K≤N
2+4+6+…+N (জোড়)	S=0, K=2	S=S+K	K=K+2	K≤N
1+3+5+…+N (বিজোড়)	S=0, K=1	S=S+K	K=K+2	K≤N
$1^2+2^2+3^2+\ldots+N^2$	S=0, K=1	S=S+(K*K)	K=K+1	K≤N
$1^2+3^2+5^2+\ldots+N^2$	S=0, K=1	S=S+(K*K)	K=K+2	K≤N
$2^2+4^2+6^2+\ldots+N^2$	S=0, K=2	S=S+(K*K)	K=K+2	K≤N
$1^3+2^3+3^3+\ldots+N^3$	S=0, K=1	S=S+(K*K*K)	K=K+1	K≤N
5+10+15+…+N	S=0, K=5	S=S+K	K=K+5	K≤N
1.5+2.5+3.5+…+N	S=0, K=1.5 (float)	S=S+K	K=K+1	K≤N
N! = 1×2×…×N	**S=1**, K=1	S=S*K	K=K+1	K≤N

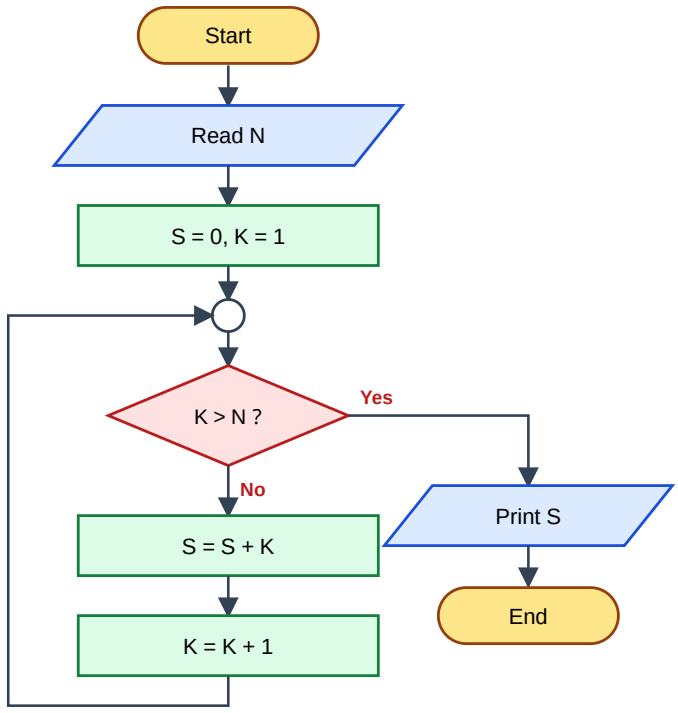
Goto-স্টাইল অ্যালগরিদম (1+2+…+N)

ধাপ-1: Start
ধাপ-2: Read N
ধাপ-3: S = 0, K = 1
ধাপ-4: যদি K > N হয়, তবে ধাপ-৭ এ যাও
ধাপ-5: নইলে S = S + K, K = K + 1
ধাপ-6: ধাপ-৪ এ যাও (goto Step4)
ধাপ-7: Print S
ধাপ-8: Stop

একই সমস্যা — সাধারণ রীতি

ধাপ-1: Start
ধাপ-2: Read N
ধাপ-3: S = 0, K = 1
ধাপ-4: S = S + K
ধাপ-5: K = K + 1
ধাপ-6: K ≤ N হলে ধাপ-৪ এ যাও
ধাপ-7: Print S
ধাপ-8: End

Flowchart (1+2+…+N)



ট্রেস টেবিল / Dry Run (N=3)

ধাপ	K>N?	S=S+K	K=K+1
শুরু	—	S=0	K=1
১ম	1>3? না	0+1=1	2
২য়	2>3? না	1+2=3	3
৩য়	3>3? না	3+3=6	4
৪র্থ	4>3? **হ্যাঁ**	থামে → **S = 6**	

⚡ সিরিজ দেখে ৩টা জিনিস বের করো

১ম পদ = K-এর শুরুর মান | **২য় পদ − ১ম পদ** = step (K কত করে বাড়বে) | বর্গ/ঘন হলে ভেতরে K*K বা K*K*K।
যেমন 5,10,15… → শুরু 5, step 10−5 = 5 → K=K+5।

সব সিরিজ এক প্রোগ্রামে (N = 10)

```c
#include <stdio.h>
int main()
{
    int N, K, S;
    scanf("%d", &N);
    S = 0;  for (K = 1; K <= N; K++)    S = S + K;
    printf("1+2+..+N      = %d\n", S);
    S = 0;  for (K = 2; K <= N; K += 2) S = S + K;
    printf("2+4+6+..      = %d\n", S);
    S = 0;  for (K = 1; K <= N; K += 2) S = S + K;
    printf("1+3+5+..      = %d\n", S);
    S = 0;  for (K = 1; K <= N; K++)    S = S + K*K;
    printf("1^2+2^2+..    = %d\n", S);
    S = 0;  for (K = 1; K <= N; K++)    S = S + K*K*K;
    printf("1^3+2^3+..    = %d\n", S);
    S = 0;  for (K = 5; K <= N; K += 5) S = S + K;
    printf("5+10+15+..    = %d\n", S);
    float F = 0, X;
    for (X = 1.5; X <= N; X = X + 1) F = F + X;
    printf("1.5+2.5+..    = %.1f\n", F);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 10
1+2+..+N      = 55
2+4+6+..      = 30
1+3+5+..      = 25
1^2+2^2+..    = 385
1^3+2^3+..    = 3025
5+10+15+..    = 15
1.5+2.5+..    = 49.5
```

যাচাই: 1+…+10 = 10×11/2 = 55 ✓; বর্গের যোগফল 10×11×21/6 = 385 ✓; ঘনের যোগফল 55^2 = 3025 ✓

৯ C প্রোগ্রামিং ভাষা — পরিচিতি ও গঠন

ইতিহাস, বৈশিষ্ট্য, গঠন, হেডার ফাইল, main

C-এর ইতিহাস ফাইনাল নোট

১৯৭২ সালে যুক্তরাষ্ট্রের **Bell Labs (AT&T Bell Laboratories)**-এ **ডেনিস রিচি (Dennis Ritchie)** C ভাষা উদ্ভাবন করেন। **DEC PDP-11** মেশিনে Unix অপারেটিং সিস্টেম C দিয়ে লেখা হয়। C একটি **Structured**, **৩য় প্রজন্মের (3GL)** উচ্চস্তরের ভাষা — আবার বিট-লেভেল কাজের জন্য একে **মিড লেভেল**-ও বলে। অনুবাদক: **Compiler**।

🧠 এক লাইনে

"**রিচি-র ৭২-এ বেল বাজে**" → Ritchie, 1972, Bell Labs।

📌 সংযোজন: পূর্বসূরি

BCPL → B (Ken Thompson) → **C** (Dennis Ritchie)। Unix প্রথমে PDP-7-এ অ্যাসেম্বলিতে লেখা হয়েছিল; ১৯৭৩-এ PDP-11-এ C দিয়ে আবার লেখা হয়।

C ভাষার বৈশিষ্ট্য ফাইনাল নোট

1. প্রোগ্রাম শুরু হয় **main()** ফাংশন থেকে।
2. স্টেটমেন্টগুলো দ্বিতীয় বন্ধনী **{ }**-এর ভেতরে।
3. প্রতিটি স্টেটমেন্টের শেষে **;** (সেমিকোলন)।
4. **Case Sensitive** — sum ও Sum আলাদা; কোড সাধারণত ছোট হাতে।
5. %d, %f ইত্যাদি ফরম্যাট স্পেসিফায়ার দিয়ে প্রদর্শন; ইচ্ছেমতো দশমিক নির্ভুলতা (%.2f)।
6. প্রচুর লাইব্রেরি ফাংশন, গ্রাফিক্স লাইব্রেরি ও কন্ট্রোল স্টেটমেন্ট।
7. **Portable** — এক মেশিনের প্রোগ্রাম অন্য মেশিনে চলে।
8. অনেক অ্যাসাইনমেন্ট অপারেটর (+=, −=, *= ...)।
9. কোডে ১-Tab ইন্ডেন্টেশন দেওয়া প্রচলিত রীতি (বাধ্যতামূলক নয়)।

৯.১ C প্রোগ্রামের গঠন — ৬টি অংশ (বই অনুযায়ী) ফাইনাল নোট

অংশ	কাজ
১. Documentation	প্রোগ্রামের নাম/বর্ণনা — কমেন্ট আকারে; নির্বাহে প্রভাব নেই।
২. Link	হেডার ফাইল যুক্ত করা — `#include<stdio.h>`
৩. Definition	Symbolic Constant নির্ধারণ — `#define PI 3.1416`
৪. Global Declaration	main ও সাব-প্রোগ্রাম — সবখানে ব্যবহারযোগ্য চলক।
৫. main() Function	Declaration part (চলক ঘোষণা) + Executable part (কাজ); এখান থেকেই রান শুরু।
৬. Subprogram	User-defined function।

🧠 ক্রম মনে রাখো

"**ডক-লিংক-ডেফ-গ্লোবাল-মেইন-সাব**" → **D L D G M S** = "**D**ad **L**ikes **D**al, **G**ood **M**orning **S**ir"

৬ অংশ একটি প্রোগ্রামে

```c
/* Documentation: বৃত্তের ক্ষেত্রফল */
#include <stdio.h>          /* Link */
#define PI 3.1416           /* Definition */
float area;                 /* Global */
float calc(float r);        /* prototype */
int main()                  /* main */
{
    float r;                    /*
declaration */
    scanf("%f", &r);            /*
executable */
    area = calc(r);
    printf("Area = %.2f", area);
    return 0;
}
float calc(float r)         /* Subprogram */
{
    return PI * r * r;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 2
Area = 12.57
```

৯.২ নোট-২ অনুযায়ী C প্রোগ্রামের গঠন — ৬টি ধাপ নোট-২

ধাপ	কী লিখব	উদাহরণ
১. Library Program	হেডার ফাইল	`#include<stdio.h>`
২. Main function	main-এর শিরোনাম	`int main()`
৩. Variable declare	Data Type সহ চলক	`int a,b,sum;`
৪. Input	scanf()	`scanf("%d %d",&a,&b);`
৫. Process	সূত্র/শর্ত/লুপ	`sum=a+b;`
৬. Output	printf()	`printf("%d",sum);`

⚡ বিল্ডিং-এর সাথে তুলনা নোট-২

Software-কে ভবনের সাথে তুলনা করা হয় — ভবন বানাতে ৩টি কাজ: **পরিকল্পনা → ম্যাপ আঁকা → বাস্তবায়ন**। প্রোগ্রামেও তেমনি: **Algorithm → Flowchart → C Coding**। Program-এর অপর নাম Software বা Application (App)।

🎯 পরীক্ষায় "C প্রোগ্রাম লেখ" এলে

এই ৬ ধাপ ক্রমানুসারে লিখলেই কাঠামো ঠিক থাকবে: **হেডার → main → চলক → scanf → সূত্র → printf**। শেষে `return 0;` ও `}` ভুলবে না।

৯.৩ হেডার ফাইল / Library Program নোট-২ + ফাইনাল নোট

Library Program (হেডার ফাইল)

কতগুলো ছোট ছোট পূর্বনির্মিত প্রোগ্রাম (ফাংশন) নিয়ে গঠিত ফাইল। `#include` দিয়ে যুক্ত করলে এর ফাংশনগুলো ব্যবহার করা যায়। ফাইলের শেষে **.h** = header।

হেডার	পূর্ণরূপ	কাজ / ফাংশন
stdio.h	**st**andard **i**nput **o**utput **h**eader	printf(), scanf() — ইনপুট/আউটপুট। প্রায় সব প্রোগ্রামে আবশ্যক।
conio.h	**con**sole **i**nput **o**utput	clrscr() — স্ক্রিন পরিষ্কার; getch() — একটি কী চাপা পর্যন্ত আউটপুট স্ক্রিন ধরে রাখে। নোট-২: "এটি ব্যবহার করলে শেষে getch() দিতে হবে"।
math.h	mathematics	pow(), sqrt(), cbrt(), fabs(), sin(), cos(), tan(), log(), log10()
string.h	string	strlen(), strcpy(), strcat(), strcmp()
stdlib.h	standard library	abs(), rand(), malloc(), exit()
graphics.h / time.h	—	গ্রাফিক্স (Turbo C) / সময়-তারিখ

⚠ সংশোধন

উৎসে: নোট-২: "graphic.h"

সঠিক: graphics.h

কেন: হেডারের নাম একটি অক্ষরও ভুল হলে Compiler "No such file" error দেয়।

⚠ conio.h সম্পর্কে সতর্কতা সংযোজন

conio.h ও getch()/clrscr() C-এর **স্ট্যান্ডার্ড অংশ নয়** — পুরনো Turbo C (DOS/Windows)-এ চলে, Linux-এর gcc-তে নেই। বোর্ডের পুরনো ধাঁচের উত্তরে লেখা চলে, কিন্তু না লিখলেও প্রোগ্রাম সঠিক।

math.h — গাণিতিক রাশি ↔ C ফাংশন

গণিতে	C-তে	নোট
a^m	`pow(a,m)`	pow(2,3)=8
√x	`sqrt(x)`	sqrt(25)=5
$\sqrt[3]{y}$	`cbrt(y)`	cbrt(27)=3
|x| (পূর্ণসংখ্যা)	`abs(x)`	stdlib.h-তে
|x| (দশমিক)	`fabs(x)`	math.h-তে
sin x, cos x, tan x	`sin(x)` …	x রেডিয়ানে
ln x / $\log_{10}$x	`log(x)` / `log10(x)`	
e^x	`exp(x)`	সংযোজন

math.h ফাংশন চালিয়ে দেখা

```c
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
int main()
{
    printf("pow(2,3)   = %.0f\n",
pow(2,3));
    printf("sqrt(25)   = %.0f\n",
sqrt(25));
    printf("cbrt(27)   = %.0f\n",
cbrt(27));
    printf("abs(-7)    = %d\n", abs(-7));
    printf("fabs(-2.5) = %.1f\n",
fabs(-2.5));
    printf("log10(100) = %.0f\n",
log10(100));
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
pow(2,3)   = 8
sqrt(25)   = 5
cbrt(27)   = 3
abs(-7)    = 7
fabs(-2.5) = 2.5
log10(100) = 2
```

⚠ সংশোধন
উৎসে: নোট-২: "|x| = abs(x)" (সব সংখ্যার জন্য) এবং "Sin, Cos, tan, Cot, Sec, Cosec" math.h-এর ফাংশন
সঠিক: abs() শুধু **int**-এর জন্য (stdlib.h); দশমিকের জন্য **fabs()**। math.h-এ আছে শুধু **sin, cos, tan** (এবং asin, acos, atan)। cot, sec, cosec ফাংশন নেই — লিখতে হয় `1/tan(x)`, `1/cos(x)`, `1/sin(x)`।
কেন: abs(-2.5) লিখলে উত্তর আসে 2 (দশমিক কেটে যায়); cot(x) লিখলে compiler error দেয়।

৯.৪ main ফাংশন — ৩ রূপ নোট-২

রূপ	অর্থ
`main()`	পুরনো রীতি; return type উল্লেখ নেই (ধরে নেয় int)
`void main()`	কিছু ফেরত দেয় না (Turbo C-তে চলে)
`int main()`	শেষে `return 0;` — "ত্রুটিহীনভাবে শেষ"। **আধুনিক ও সবচেয়ে সঠিক রূপ**

⚠ সংশোধন
উৎসে: নোট-২: "viod main()"
সঠিক: **void** main()
কেন: বানান ভুল (viod) হলে Syntax Error।

প্রথম C প্রোগ্রাম

```c
#include <stdio.h>
int main()
{
    printf("Hello world");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
Hello world
```

stdio.h → হেডার; main() → শুরু; printf() → স্ক্রিনে লেখা; return 0 → ত্রুটি ছাড়া শেষ

৯ক ডেটা টাইপ, চলক, ধ্রুবক

মেমোরি সাইজ, ফরম্যাট স্পেসিফায়ার, নামকরণের নিয়ম

Data Type

চলকে কোন ধরনের ডেটা (পূর্ণসংখ্যা, দশমিক, অক্ষর) থাকবে এবং মেমোরিতে কত জায়গা লাগবে — তা নির্দেশ করে।

Data Type-এর শ্রেণিবিভাগ (৩ প্রকার)	নোট-২ + ফাইনাল নোট
১. Built-in / Basic / Primary / মৌলিক	char, int, float, double (+ void)
২. User-defined	ব্যবহারকারী নিজে তৈরি করে — structure, union, enum, typedef
৩. Derived	মৌলিক টাইপ থেকে তৈরি — Array, Pointer, Function

🧠 Derived বোঝার সহজ উদাহরণ হাতে-লেখা নোট

মৌলিক তথ্য: জন্মসাল = 2000 → উদ্ভূত (Derived) তথ্য: বয়স = 18। তেমনি int থেকে তৈরি হয় **int A[10]** (Array) — এটি Derived।

Data type	বোর্ড বই (Turbo C)	আধুনিক gcc	Format	উদাহরণ
char	1 byte (8 bit)	1 byte	%c	'A', '$'
int	**2 byte** (16 bit)	4 byte	%d	10, −5000
float	4 byte (32 bit)	4 byte	%f	10.123
double	8 byte (64 bit)	8 byte	%lf	বড়/সূক্ষ্ম দশমিক

sizeof দিয়ে যাচাই (gcc)

```c
#include <stdio.h>
int main()
{
    printf("char   = %d byte\n",
(int)sizeof(char));
    printf("int    = %d byte\n",
(int)sizeof(int));
    printf("float  = %d byte\n",
(int)sizeof(float));
    printf("double = %d byte\n",
(int)sizeof(double));
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: —
char   = 1 byte
int    = 4 byte
float  = 4 byte
double = 8 byte
```

⚠ সংশোধন

উৎসে: নোট-২ এক জায়গায় int = 4 byte, অন্য জায়গায় ও ফাইনাল নোটে int = 2 byte

সঠিক: **বোর্ড পরীক্ষায় int = 2 byte লিখবে** (বইয়ের Turbo C মান)। বাস্তবে আধুনিক gcc-তে int = 4 byte — উপরের প্রোগ্রাম চালিয়ে তা-ই দেখা গেল।

কেন: int-এর আকার কম্পাইলার/মেশিন-নির্ভর; C স্ট্যান্ডার্ড শুধু বলে int ন্যূনতম 2 byte। দুই নোট দুই কম্পাইলারের কথা বলছে — কেউই পুরো ভুল নয়, কিন্তু পরীক্ষায় বইয়ের মানটাই নিরাপদ।

⚠ সংশোধন

উৎসে: ফাইনাল নোট: double = "৩৭০০০+ অঙ্ক পর্যন্ত ভগ্নাংশ"

সঠিক: double-এর নির্ভুলতা মোটামুটি **১৫–১৬টি অর্থবহ অঙ্ক**; পরিসর প্রায় **$\pm 1.7\times10^{308}$**। float-এর নির্ভুলতা ~৬–৭ অঙ্ক।

কেন: ৩৭০০০ অঙ্ক কোনো ডেটা টাইপে থাকে না — সম্ভবত পরিসরের সংখ্যাটি ভুলভাবে লেখা হয়েছে।

চলক (Variable) ও ধ্রুবক (Constant)

চলক (Variable)

প্রোগ্রামে যে ডেটার মান **পরিবর্তনশীল**, অর্থাৎ মেমোরির যে নামকৃত জায়গায় মান রাখা হয় ও বদলানো যায়। `int a; a = 10; a = a + 5;`

ঘোষণার নিয়ম নোট-২ : দুইটা পূর্ণসংখ্যা → `int x, y;`
তিনটা ভগ্নাংশ → `float x, y, z;`

ধ্রুবক (Constant)

প্রোগ্রাম চলাকালে যার মান **অপরিবর্তিত** থাকে। যেমন 3.1416, 5, 'A'।

C-তে ঘোষণা: `#define PI 3.1416` বা `const float PI = 3.1416;`

⚠ ছোট সংশোধন সংযোজন

ফাইনাল নোটে ধ্রুবকের উদাহরণ "`Pie = 3.1416; x = 5;`" — এভাবে লিখলে Pie আসলে একটি **চলক**, যার মান পরে বদলানো যায়। সত্যিকারের ধ্রুবক বানাতে `#define` বা `const` লাগে।

চলক নামকরণের ৬ নিয়ম (MCQ-তে বারবার আসে) ফাইনাল নোট	
১	প্রথম অক্ষর হবে বর্ণ (A-Z, a-z) বা আন্ডারস্কোর (_) — **সংখ্যা দিয়ে শুরু নয়**
২	শুধু A-Z, a-z, 0-9 ও _ ব্যবহার করা যাবে
৩	স্পেস, বিন্দু (.), $, -, # ইত্যাদি নিষেধ
৪	Keyword/রিজার্ভড শব্দ (int, main, return, if…) নাম হবে না
৫	Case Sensitive — X ও x আলাদা চলক
৬	নাম সর্বোচ্চ ৩১ অক্ষর পর্যন্ত অর্থবহ ধরা হয়

নাম	বৈধ?	কারণ
Std_1	✓	নিয়ম মেনেছে
_total	✓	_ দিয়ে শুরু চলে
1std	×	সংখ্যা দিয়ে শুরু
Std.1	×	বিন্দু
my name	×	স্পেস
int	×	keyword
$sum	×	$ চিহ্ন

🧠 **"অক্ষর/_ দিয়ে শুরু, মাঝে অক্ষর-সংখ্যা-_ ; স্পেস-চিহ্ন-keyword মানা"**

৯খ ইনপুট-আউটপুট: scanf(), printf(), Format Specifier

Escape sequence, Comment

Format Specifier নোট-২

ইনপুট নেওয়া বা আউটপুট দেওয়ার সময় চলকের Data Type বোঝাতে যে বিশেষ সংকেত ব্যবহৃত হয় (% দিয়ে শুরু)। int → **%d**, float → **%f**, char → **%c**, double → **%lf**, string → **%s**।

scanf() — ইনপুট

গঠন: `scanf("Format Specifier", &variable);`

& = চলকের **ঠিকানা (address)**; scanf সেই ঠিকানায় মান জমা রাখে। তাই scanf-এ চলকের আগে & **সবসময়**।

উদা.: ১টি int ও ১টি float → `scanf("%d %f", &a, &x);`

printf() — আউটপুট

গঠন: `printf("Format Specifier", variable);` — এখানে & লাগে না।

উদা.: `printf("%d %f", a, x);`

দশমিকের পর ২ ঘর: `%.2f`

I/O ও escape sequence

```c
#include <stdio.h>
int main()
{
    int a;  float x;  char c;
    scanf("%d %f %c", &a, &x, &c);
    printf("a = %d\n", a);
    printf("x = %f\n", x);
    printf("x = %.2f\n", x);
    printf("c = %c\n", c);
    printf("Tab:\tA\tB\n");
    printf("Quote: \"Hi\"  Slash: \\");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 25 3.14159 Z
a = 25
x = 3.141590
x = 3.14
c = Z
Tab:    A       B
Quote: "Hi"  Slash: \
```

⚠ সংশোধন

উৎসে: নোট-২: "'%' এটিকে Modulus বা Absolute value বলে"

সঠিক: ফরম্যাট স্ট্রিং-এর ভেতরে % হলো **Format specifier-এর চিহ্ন** (%d, %f)। অপারেটর হিসেবে `a % b` হলো **Modulus = ভাগশেষ**। Absolute value (|x|) হলো abs()/fabs() — % নয়।

কেন: একই চিহ্নের দুটি ভিন্ন ব্যবহার; Absolute value-এর সাথে % এর কোনো সম্পর্ক নেই।

Escape Sequence ফাইনাল নোট

`\n`	নতুন লাইন
`\t`	Tab (ফাঁকা ঘর) — সংযোজন
`\"`	ডাবল কোটেশন দেখাতে
`\\`	ব্যাকস্ল্যাশ দেখাতে
`\0`	Null character — স্ট্রিং-এর শেষ চিহ্ন

Comment (মন্তব্য)

প্রোগ্রামের ব্যাখ্যা; Compiler এড়িয়ে যায়, নির্বাহে প্রভাব নেই।

Single line: `// ...`

Multi line: `/* ... */`

⚡ শর্টকাট

scanf মান রাখে → & লাগে; printf মান দেখায় → & লাগে না।

৯গ অপারেটর ও এক্সপ্রেশন

গাণিতিক ↔ C এক্সপ্রেশন, ++/--, বিটওয়াইজ শিফট

Operator, Operand, Expression

ফাইনাল নোট

`Sum = a + b` — **+** = Operator (চিহ্ন), **a, b** = Operand (যার উপর কাজ), পুরোটা = **Expression**।

operand সংখ্যা অনুযায়ী ৩ প্রকার

Unary	১টি	-a, ++i, i--, !x
Binary	২টি	a+b, a>b
Ternary	৩টি	`(a>b) ? a : b`

ধরন	অপারেটর
Arithmetic	+ − * / %
Relational	< <= > >= **==** (সমান) != (অসমান)
Logical	&& (AND) || (OR) ! (NOT)
Assignment	= += −= *= /= %=
Inc/Dec	++ --
Bitwise	& | ^ ~ << >>

অপারেটর চালিয়ে দেখা

```c
#include <stdio.h>
int main()
{
    int a = 17, b = 5, i = 5, x;
    printf("a/b = %d\n", a / b);
    printf("a%%b = %d\n", a % b);
    printf("17.0/5 = %.1f\n", 17.0 / 5);
    x = i++;  printf("x=i++ : x=%d i=%d\n", x,
i);
    i = 5;
    x = ++i;  printf("x=++i : x=%d i=%d\n", x,
i);
    printf("10<<1 = %d, 10>>1 = %d\n", 10<<1,
10>>1);
    printf("max = %d\n", (a > b) ? a : b);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
a/b = 3
a%b = 2
17.0/5 = 3.4
x=i++ : x=5 i=6
x=++i : x=6 i=6
10<<1 = 20, 10>>1 = 5
max = 17
```

Pre vs Post Increment

i++ (Post): আগে পুরনো মান ব্যবহার, পরে ১ বাড়ে → x=5, i=6।

++i (Pre): আগে ১ বাড়ে, তারপর ব্যবহার → x=6, i=6। একইভাবে i-- ও --i (১ কমে)।

Modulus (%) ও ভাগ (/)

int/int = পূর্ণসংখ্যা অংশ: 17/5 = **3**; ভাগশেষ: 17%5 = **2**; 10%3 = 1; 50%5 = 0। % শুধু পূর্ণসংখ্যায় চলে।

বিটওয়াইজ শিফট

<< n = ×2^n, **>> n** = ÷2^n

00001010 (10) <<1 → 00010100 (**20**)

00001010 (10) >>1 → 00000101 (**5**)

A	B	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

🧠

AND = দুটোই ১ → ১; OR = যেকোনো ১ → ১; XOR = ভিন্ন হলে ১। "<< গুণ, >> ভাগ"।

গাণিতিক রাশি ↔ C Expression নোট-২

গণিতে	C-তে
× বা ·	*
÷	/ (slash)
a^m	pow(a,m)
√x	sqrt(x)
$\sqrt[3]{y}$	cbrt(y)
|x|	abs(x) / fabs(x)
ভাগশেষ (mod)	%
= (তুলনা)	==
≠ , ≤ , ≥	!= , <= , >=

রাশি	C Expression
½·b·h	0.5*b*h
πr²	3.1416*r*r
(√3/4)a²	sqrt(3)/4*a*a
(b/4)√(4a²−b²)	b/4.0*sqrt(4*a*a-b*b)
(−b+√(b²−4ac))/2a	(-b+sqrt(b*b-4*a*c))/(2*a)
9C/5+32	9*c/5+32

⚠ নিয়ম নোট-২

C Expression সাধারণত **ছোট হাতের অক্ষরে** (sqrt, pow, printf — বড় হাতে লিখলে চলবে না)। গুণ চিহ্ন কখনো বাদ দেওয়া যাবে না: 2a নয়, **2*a**। ভগ্নাংশে বন্ধনী: **(2*a)**।

৯ঘ শর্তমূলক স্টেটমেন্ট: if, if-else, else-if ladder, nested, switch

Process ধাপের প্রথম অস্ত্র

Conditional Statement নোট-২ + ফাইনাল নোট

শর্ত (condition) **সত্য** হলে এক কাজ, **মিথ্যা** হলে অন্য কাজ — এই সিদ্ধান্ত নেওয়ার স্টেটমেন্ট। নোট-২ অনুযায়ী Process ধাপে ৩টি জিনিস থাকে: **গাণিতিক সূত্র, Conditional statement (if, else, else-if), Looping statement (for, while, do-while)**।

রূপ	গঠন	কখন
if	`if(শর্ত){ ... }`	শুধু সত্য হলে কিছু করতে
if-else	`if(শর্ত){ ... } else { ... }`	দুই পথের একটি
else-if ladder	`if(c1){} else if(c2){} else if(c3){} else{}`	ধারাবাহিক বহু শর্ত (গ্রেড)
Nested if	if-এর ভেতরে আরেক if-else	শর্তের ভেতরে শর্ত (৩ সংখ্যার বৃহত্তম)
switch	`switch(x){ case 1: ...; break; default: ... }`	একটি চলকের নির্দিষ্ট মান অনুযায়ী

else-if ladder: পূর্ণ গ্রেডিং পদ্ধতি

```c
#include <stdio.h>
int main()
{
    int number;
    scanf("%d", &number);
    if (number > 100 || number < 0)
        printf("Invalid");
    else if (number >= 80) printf("A+ (GPA 5.00)");
    else if (number >= 70) printf("A  (GPA 4.00)");
    else if (number >= 60) printf("A- (GPA 3.50)");
    else if (number >= 50) printf("B  (GPA 3.00)");
    else if (number >= 40) printf("C  (GPA 2.00)");
    else if (number >= 33) printf("D  (GPA 1.00)");
    else                   printf("F  (Fail)");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 64
A- (GPA 3.50)
```

⚠ সংশোধন

উৎসে: নোট/ফাইনাল নোটের গ্রেড উদাহরণ: "≥80 → A+, ≥70 → A, ≥50 → A-, নইলে Fail"

সঠিক: বাংলাদেশের প্রকৃত গ্রেডিং: 80–100 A+, 70–79 A, **60–69 A-**, 50–59 B, 40–49 C, 33–39 D, 0–32 F (উপরের কোড)।

কেন: উদাহরণটি ladder শেখানোর জন্য সংক্ষিপ্ত; কিন্তু "50 পেলেই A-" বাস্তবে ভুল। বোর্ডে গ্রেডিং প্রশ্ন এলে পূর্ণ ছক ব্যবহার করবে।

রূপ-১: else-if ladder + &&

```c
if (A > B && A > C)
    printf("%d", A);
else if (B > A && B > C)
    printf("%d", B);
else
    printf("%d", C);
```

রূপ-২: Nested if-else

```c
if (A > B) {
    if (A > C) printf("%d", A);
    else       printf("%d", C);
}
else {
    if (B > C) printf("%d", B);
    else       printf("%d", C);
}
```

🎯 তিনটি সংখ্যার বৃহত্তম — দুই রূপই সঠিক

বোর্ডে যেকোনোটি লিখলেই পূর্ণ নম্বর। ক্ষুদ্রতমের জন্য শুধু **> এর জায়গায় <**।

⚠ = বনাম ==

`if(a = 5)` ভুল — এটা a-তে 5 বসিয়ে দেয় এবং সবসময় সত্য! তুলনার জন্য **==**: `if(a == 5)` ।

switch: সপ্তাহের দিন (break সহ)

```c
#include <stdio.h>
int main()
{
    int day;
    scanf("%d", &day);
    switch (day)
    {
        case 1: printf("Saturday");  break;
        case 2: printf("Sunday");    break;
        case 3: printf("Monday");    break;
        case 4: printf("Tuesday");   break;
        case 5: printf("Wednesday"); break;
        case 6: printf("Thursday");  break;
        case 7: printf("Friday");    break;
        default: printf("Invalid");
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 2
Sunday
```

break ছাড়া — Fall-through

```c
#include <stdio.h>
int main()
{
    int day = 2;
    switch (day)
    {
        case 1: printf("Saturday ");
        case 2: printf("Sunday ");
        case 3: printf("Monday ");
        default: printf("Invalid");
    }
    return 0;
}
```

⚠ Fall-through ফাঁদ

break না দিলে মিলে যাওয়া case থেকে **নিচের সব case** চলতে থাকে — পাশের আউটপুটে "Sunday Monday Invalid" এসেছে, যদিও শুধু Sunday আসার কথা।

⚡ switch-এর নিয়ম

switch-এর expression হবে int বা char (float নয়); case-এর মান ধ্রুবক; **default** ঐচ্ছিক — কোনো case না মিললে চলে।

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
Sunday Monday Invalid
```

৯ঙ লুপ (Loop): while, do-while, for, break, continue

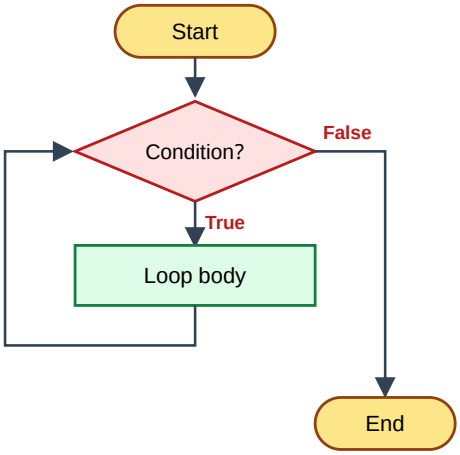
পুনরাবৃত্তি

Loop নোট-২ + ফাইনাল নোট

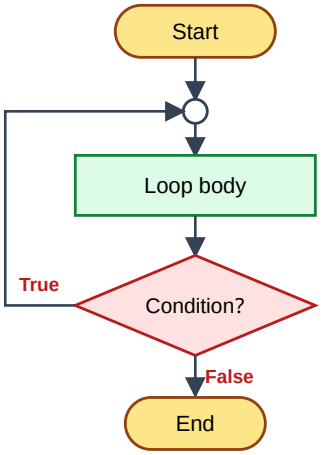
কোনো কাজ নির্দিষ্ট শর্ত সাপেক্ষে **বারবার** করার পদ্ধতি। প্রতিটি লুপে ৩টি অংশ আবশ্যক: **(i) Counter-এর শুরুর মান (ii) Condition (iii) Counter বৃদ্ধি/হ্রাস (++/--)**। Loop ৩ প্রকার: for, while, do-while।

লুপ	গঠন	বৈশিষ্ট্য
while	`init; while(condition){ body; update; }`	শর্ত **আগে** যাচাই — শুরুতেই মিথ্যা হলে **০ বার** চলে (Entry-controlled)
do-while	`init; do{ body; update; }while(condition);`	body আগে, শর্ত **পরে** — **কমপক্ষে ১ বার** চলবেই (Exit-controlled); শেষে **;** আবশ্যক
for	`for(init; condition; update){ body; }`	৩ অংশ এক লাইনে, ঠিক ২টি সেমিকোলন; গণনা-নির্ভর কাজে সবচেয়ে বেশি ব্যবহৃত

while loop — আগে প্রশ্ন, পরে কাজ



do-while loop — আগে কাজ, পরে প্রশ্ন



একই সমস্যা (1+2+…+N) তিন লুপে

ফাইনাল নোট

while

```c
// while loop
int i = 1, Sum = 0;
while (i <= n)
{
    Sum = Sum + i;
    i++;
}
printf("%d", Sum);
```

do-while

```c
// do-while loop
int i = 1, Sum = 0;
do
{
    Sum = Sum + i;
    i++;
} while (i <= n);
printf("%d", Sum);
```

for

```c
// for loop
int Sum = 0;
for (int i = 1; i <= n; i++)
{
    Sum = Sum + i;
}
printf("%d", Sum);
```

🧠 **পার্থক্য এক নজরে**

মূল কাজ (`Sum=Sum+i; i++`) তিনটিতেই একই, শুধু **কোথায় বসছে** তা ভিন্ন: for-এ init/condition/update এক লাইনে; while-এ শর্ত আগে; do-while-এ শর্ত শেষে।

শর্ত শুরুতেই মিথ্যা (i=10 < 5?) — কোনটি চলে?

```c
#include <stdio.h>
int main()
{
    int i = 10;
    while (i < 5)
    {
        printf("while ran\n");
        i++;
    }
    i = 10;
    do
    {
        printf("do-while ran once (i=%d)\n", i);
        i++;
    } while (i < 5);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
do-while ran once (i=10)
```

while একবারও চলেনি; do-while ঠিক ১ বার চলেছে।

Infinite Loop (অসীম লুপ)

যে লুপের শর্ত সবসময় সত্য থাকে, ফলে কখনো শেষ হয় না। উদা.: `while(1){ printf("Hello"); }` বা `for(;;)`। Update (i++) ভুলে গেলেও লুপ অসীম হয়ে যায়।

break	continue
লুপ থেকে **সম্পূর্ণ বের** হয়ে যায়	শুধু **বর্তমান ধাপ বাদ** দিয়ে পরের ধাপে যায়
switch-এও ব্যবহৃত হয়	শুধু লুপে

🧠

"Break = দরজা দিয়ে বের হওয়া, Continue = এক ধাপ টপকানো"

break

```c
#include <stdio.h>
int main()
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 5) break;
        printf("%d ", i);
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
0 1 2 3 4
```

continue

```c
#include <stdio.h>
int main()
{
    for (int i = 0; i < 10; i++)
    {
        if (i == 5) continue;
        printf("%d ", i);
    }
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: –
0 1 2 3 4 6 7 8 9
```

একটি পূর্ণ প্রোগ্রাম — প্রতিটি লাইনের ব্যাখ্যা (1 থেকে N যোগফল) ফাইনাল নোট

কোড-লাইন	ব্যাখ্যা
`#include<stdio.h>`	হেডার ফাইল — printf, scanf ব্যবহারের জন্য।
`main()`	মূল ফাংশন — এখান থেকে কাজ শুরু (Start)।
`int N,K,S;`	তিনটি পূর্ণসংখ্যা চলক ঘোষণা (Declaration part)।
`scanf("%d",&N);`	কীবোর্ড থেকে N নেওয়া; &N = N-এর ঠিকানা।
`S=0;`	যোগফলের শুরুর মান শূন্য।

`for(K=1;K<=N;K=K+1)`	শুরু K=1, শর্ত K≤N, প্রতিবার K এক করে বাড়ে।
`{ S=S+K; }`	লুপের মূল কাজ — প্রতিবার K যোগ হয়ে নতুন S।
`printf("%d",S);`	চূড়ান্ত ফল দশমিক আকারে স্ক্রিনে।
`}`	প্রোগ্রামের সমাপ্তি (Stop)।

🧠 **যেকোনো C কোড পড়ার ৮-ধাপ ক্রম** ফাইনাল নোট

হেডার → main শুরু → চলক ঘোষণা → ইনপুট → প্রাথমিক মান → লুপ/শর্ত → প্রসেসিং → আউটপুট → শেষ

⚠ **সংশোধন**

উৎসে: ক্রম গুলিয়ে do-while লেখা: `K=1; while(K<=N) do { ... }`

সঠিক: `K=1; do { S=S+K; K=K+1; } while(K<=N);` — init লুপের আগে, শর্ত শেষে, শেষে সেমিকোলন।

কেন: শর্ত আগে বসালে do-while-এর মূল বৈশিষ্ট্য (কমপক্ষে ১ বার) নষ্ট হয় এবং Syntax error হয়।

৯চ ফাংশন (Function), অ্যারে (Array) ও স্ট্রিং (String)

নোট-২ + ফাইনাল নোট

Function

কোনো নির্দিষ্ট কাজের জন্য লেখা স্বয়ংসম্পূর্ণ কোড-ব্লক, যা প্রয়োজনমতো বারবার **ডাকা (call)** যায়।

প্রকার	বিবরণ	উদাহরণ
Library Function	আগে থেকে তৈরি; হেডার include করলেই ব্যবহার	printf(), scanf(), sqrt(), pow()
User-defined	প্রোগ্রামার নিজে তৈরি করেন	hello_function(), fact()

গঠন (Syntax)

```
return_type function_name(parameters)
{
    // function body
    return value;
}
```

🧠 void

"**void = ফাঁকা হাতে ফেরা**" — কোনো মান ফেরত দেয় না, শুধু কাজ করে। int/float টাইপের ফাংশন **return** দিয়ে মান ফেরত দেয়।

void ও int ফাংশন

```c
#include <stdio.h>
void hello_function()
{
    printf("Hello world\n");
}
int fact(int n)
{
    int f = 1, i;
    for (i = 1; i <= n; i++)
        f = f * i;
    return f;
}
int main()
{
    int n;
    scanf("%d", &n);
    hello_function();
    printf("%d! = %d", n, fact(n));
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 6
Hello world
6! = 720
```

Array

একই ধরনের (সমজাতীয়) একাধিক ডেটার সংগ্রহ, যা একটি নামে ক্রমিক মেমোরি ঘরে থাকে। Array হলো **Derived Data Type**।

গঠন: `Datatype Array_name[size];`

`int A[50];` → ৫০টি পূর্ণসংখ্যা; `char A[20];` → ২০টি অক্ষর।

12	7	45	3	19	8	30	5	22	11
[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

int A[10] → index **0 থেকে 9**; A[10] বলে কোনো ঘর নেই!

১০টি সংখ্যার সর্ববৃহৎ + স্ট্রিং + 2D

```c
#include <stdio.h>
int main()
{
    int numbers[10];
    for (int i = 0; i < 10; i++)
        scanf("%d", &numbers[i]);
    int max = numbers[0];
    for (int i = 0; i < 10; i++)
    {
        if (numbers[i] > max)
            max = numbers[i];
    }
    printf("max is %d\n", max);
    char s[20] = "Hello World";
    printf("%s\n", s);
    int m[2][3] = {{1,2,3},{4,5,6}};
    printf("m[1][2] = %d", m[1][2]);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 12 7 45 3 19 8 30 5 22 11
max is 45
Hello World
m[1][2] = 6
```

2D Array (দ্বিমাত্রিক)

row ও column — দুই মাত্রার অ্যারে। `int matrix[2][3];` → ২ সারি × ৩ কলাম = ৬টি ঘর।

String

Character-এর Array হলো String। যেমন `char s[20] = "Hello World";` শেষে অদৃশ্য **'\0'** (null character) বসে, যা স্ট্রিং-এর শেষ বোঝায়। Format specifier **%s**।

⚡ **ধারণা**

১ম লুপে ১০টি সংখ্যা ইনপুট; ২য় লুপে প্রতিটিকে max-এর সাথে তুলনা করে বড়টা max-এ রাখা। ক্ষুদ্রতম চাইলে শুধু > এর বদলে <।

১০ নোট-২-এর A-F-C সমস্যা (Algorithm + Flowchart + C)

প্রশ্ন ০১–০৪ + তাপমাত্রা ও ত্রিভুজ — ভুলগুলো চালিয়ে দেখানো

🎯 "A, F ও C লেখ" মানে

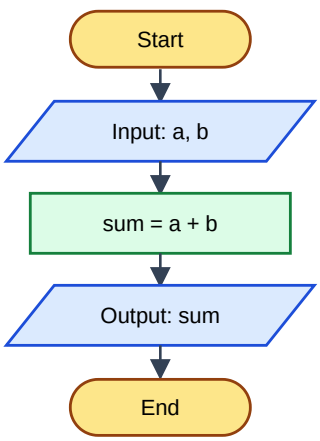
A = Algorithm, **F** = Flowchart, **C** = C Program Coding — তিনটিই লিখতে হবে। নোট-২-এর Algorithm-এর মৌলিক ৫ ধাপ: **Start → Input → Process → Output → End**।

১০.১. প্রশ্ন ০১: দুইটি পূর্ণসংখ্যার যোগফল নোট-২

Algorithm

ধাপ-1: Start
ধাপ-2: Input হিসেবে দুইটি সংখ্যা a, b গ্রহণ
ধাপ-3: sum = a + b প্রসেস
ধাপ-4: Output হিসেবে sum প্রদর্শন
ধাপ-5: End

Flowchart



C কোড (সংযোজন — পরীক্ষায় "C লেখ" এলে)

```c
#include <stdio.h>
void main()
{
    int a, b, sum;
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("%d", sum);
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 12 30
42
```

"একইভাবে বিয়োগফল, গুণফল, ভাগফল নির্ণয়ের A, F ও C লেখ" নোট-২

কাজ	Process ধাপ	সতর্কতা
বিয়োগফল	sub = a − b	—
গুণফল	mul = a * b	× নয়, *
ভাগফল	div = (float)a / b	b ≠ 0 হতে হবে; int/int হলে দশমিক কেটে যায়
ভাগশেষ	rem = a % b	শুধু int

Algorithm (একসাথে)

ধাপ-1: Start
ধাপ-2: Input a, b
ধাপ-3: sub = a − b, mul = a * b, div = a / b
ধাপ-4: Output sub, mul, div
ধাপ-5: End

বিয়োগ, গুণ, ভাগ

```c
#include <stdio.h>
int main()
{
    int a, b, sub, mul;
    float div;
    scanf("%d %d", &a, &b);
    sub = a - b;
    mul = a * b;
    div = (float)a / b;
    printf("sub = %d\n", sub);
    printf("mul = %d\n", mul);
    printf("div = %.2f\n", div);
    printf("int div (bhul) = %d", a / b);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

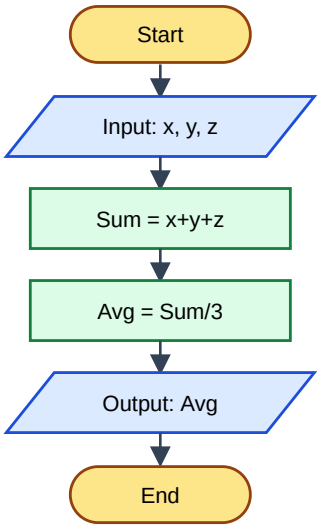
```
ইনপুট দিলাম: 7 2
sub = 5
mul = 14
div = 3.50
int div (bhul) = 3
```

১০.২. প্রশ্ন ০২: তিনটি পূর্ণসংখ্যার গড় নোট-২

Algorithm

ধাপ-1: Start
ধাপ-2: Input হিসেবে তিনটি সংখ্যা x, y, z গ্রহণ
ধাপ-3: Sum = x + y + z প্রসেস
ধাপ-4: Average = Sum / 3 প্রসেস
ধাপ-5: Output-এ Average-এর মান প্রদর্শন
ধাপ-6: End

Flowchart



নোট-২-এর হুবহু কোড ×

```c
#include <stdio.h>
void main()
{
    int x, y, z, sum;
    float avg;
    scanf("%d %d %d", &x, &y, &z);
    sum = x + y + z;
    avg = sum / 3;
    printf("%f", avg);
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 4 5 7
5.000000
```

সংশোধিত কোড ✓

```c
#include <stdio.h>
int main()
{
    int x, y, z, sum;
    float avg;
    scanf("%d %d %d", &x, &y, &z);
    sum = x + y + z;
    avg = sum / 3.0;
    printf("%f", avg);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 4 5 7
5.333333
```

⚠ সংশোধন

উৎসে: নোট-২: `avg = sum/3;` (sum হলো int)

সঠিক: `avg = sum/3.0;` অথবা `avg = (float)sum/3;`

কেন: int ÷ int = int — 16/3 = 5 (দশমিক কেটে যায়), তারপর float-এ 5.000000 রাখা হয়। আসল গড় 5.333333। উপরের দুই আউটপুট মিলিয়ে দেখো। এটি একটি **Logical Error** — প্রোগ্রাম চলে কিন্তু ফল ভুল।

১০.৩ তাপমাত্রা সম্পর্কিত Programming নোট-২

তাপমাত্রার ৩ স্কেল ও মূল সূত্র

সেলসিয়াস/সেন্টিগ্রেড = C, ফারেনহাইট = F, কেলভিন = K

$$C/5 = (F-32)/9 = (K-273)/5$$

এই একটি সূত্র থেকেই ৬ ধরনের প্রশ্নের সমাধান।

রূপান্তর	সূত্র (C Expression)
C → F	F = 9*C/5 + 32
C → K	K = C + 273
F → C	C = 5*(F−32)/9
F → K	K = 5*(F−32)/9 + 273
K → C	C = K − 273
K → F	F = 9*(K−273)/5 + 32

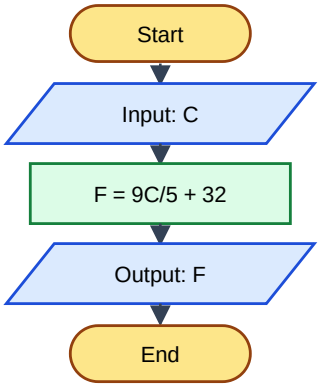
⚡ প্রতিপাদন (C → F) — রাফ খাতায়

C/5 = (F−32)/9 ⇒ 5(F−32) = 9C ⇒ F − 32 = 9C/5 ⇒ **F = 9C/5 + 32**।

নোট-২-এর পরামর্শ: পরীক্ষার খাতায় রূপান্তর অংশ না লিখে রাফে করবে; খাতায় শুধু A, F, C।

প্রশ্ন ০৩: C → F Algorithm (নোট-২)

ধাপ-1: Start
ধাপ-2: Input C
ধাপ-3: F = 9C/5 + 32 প্রসেস
ধাপ-4: Output-এ F
ধাপ-5: End



⚠ ফাঁদ

`F = 9/5*C + 32` লিখলে 9/5 = 1 (int ভাগ) → ভুল! আগে গুণ করো (`9*C/5`) বা `9.0/5`। কেলভিনে সূক্ষ্ম মান 273.15, বোর্ডে 273।

৬টি রূপান্তর একসাথে

```c
#include <stdio.h>
int main()
{
    float c, f, k;
    scanf("%f", &c);
    f = ((9 * c) / 5) + 32;
    k = c + 273;
    printf("C=%.1f -> F=%.1f, K=%.1f\n", c, f,
k);
    scanf("%f", &f);
    c = 5 * (f - 32) / 9;
    printf("F=%.1f -> C=%.1f, K=%.1f\n", f, c, c
+ 273);
    scanf("%f", &k);
    c = k - 273;
    printf("K=%.1f -> C=%.1f, F=%.1f", k, c,
9*c/5 + 32);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

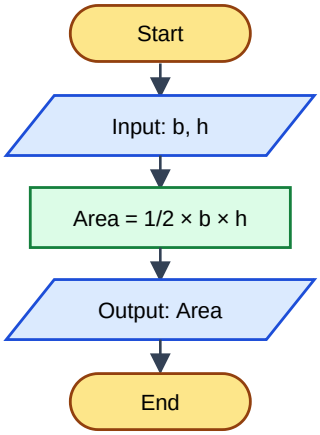
```
ইনপুট দিলাম: 37 98.6 300
C=37.0 -> F=98.6, K=310.0
F=98.6 -> C=37.0, K=310.0
K=300.0 -> C=27.0, F=80.6
```

১০.৪. প্রশ্ন ০৪: ত্রিভুজের ক্ষেত্রফল (ভূমি ও উচ্চতা) নোট-২

Algorithm

ধাপ-1: Start
ধাপ-2: Input ভূমি (b) এবং উচ্চতা (h)
ধাপ-3: Area = ½ × b × h প্রসেস
ধাপ-4: Output-এ Area প্রদর্শন
ধাপ-5: End

Flowchart



নোট-২-এর হুবহু কোড ×

```c
#include <stdio.h>
main()
{
    float b, h, area;
    scanf("%f %f", &b, &h);
    area = (1/2) * b * h;
    printf("area of the triangle is %f", area);
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 10 6
area of the triangle is 0.000000
```

সংশোধিত কোড ✓

```c
#include <stdio.h>
int main()
{
    float b, h, area;
    scanf("%f %f", &b, &h);
    area = 0.5 * b * h;
    printf("area of the triangle is %f", area);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 10 6
area of the triangle is 30.000000
```

⚠ সংশোধন

উৎসে: নোট-২: `area = (1/2)*b*h;`

সঠিক: `area = 0.5*b*h;` বা `(1.0/2)*b*h` বা `b*h/2`

কেন: C-তে 1/2 দুটোই int, তাই ফল **0** (দশমিক বাদ)। ফলে যেকোনো b, h দিলেও ক্ষেত্রফল 0.000000 — উপরে চালিয়ে প্রমাণিত। অ্যালগরিদম/ফ্লোচার্টে ½ লেখা চলে, কিন্তু C কোডে নয়।

১০.৫ ত্রিভুজ ও বৃত্তের ক্ষেত্রফলের সূত্র নোট-২

আকৃতি	গণিতে	C Expression
সাধারণ ত্রিভুজ	½ × b × h	0.5*b*h
সমবাহু ত্রিভুজ	(√3/4) a^2	sqrt(3)/4*a*a
সমদ্বিবাহু ত্রিভুজ	(b/4)√(4a^2 − b^2)	b/4*sqrt(4*a*a-b*b)
বিষমবাহু (হেরন)	√(s(s−a)(s−b)(s−c)), **s = (a+b+c)/2**	sqrt(s*(s-a)*(s-b)*(s-c))
বৃত্ত	πr^2	3.1416*r*r

⚠ সংশোধন

উৎসে: নোট-২: হেরনের সূত্রে s = (a+b+c)/**3**

সঠিক: s = (a+b+c)/**2** — অর্ধপরিসীমা (semi-perimeter)

কেন: s মানে পরিসীমার অর্ধেক। /3 দিলে 3,4,5 ত্রিভুজে ক্ষেত্রফল 6 না এসে ভুল আসে (পাশে চালিয়ে দেখো)।

সব সূত্র চালিয়ে যাচাই

```c
#include <stdio.h>
#include <math.h>
int main()
{
    float a = 3, b = 4, c = 5, s, s3;
    s  = (a + b + c) / 2;
    s3 = (a + b + c) / 3;
    printf("Heron s=/2 : %.2f\n",
        sqrt(s*(s-a)*(s-b)*(s-c)));
    printf("Heron s=/3 : %.2f (bhul)\n",
        sqrt(s3*(s3-a)*(s3-b)*(s3-c)));
    float x = 6;
    printf("Somobahu (a=6): %.2f\n",
sqrt(3)/4*x*x);
    float p = 5, q = 6;   /* a=5, b=6 */
    printf("Somodwibahu: %.2f\n", q/4*sqrt(4*p*p-
q*q));
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: —
Heron s=/2 : 6.00
Heron s=/3 : -0.00 (bhul)
Somobahu (a=6): 15.59
Somodwibahu: 12.00
```

১০.৬ ধনাত্মক, শূন্য নাকি ঋণাত্মক — C কোড ফাইনাল নোট

অংশ ৮-এর ফ্লোচার্টের C রূপ

```c
#include <stdio.h>
int main()
{
    int a;
    scanf("%d", &a);
    if (a > 0)
        printf("positive\n");
    else if (a == 0)
        printf("zero\n");
    else
        printf("negative\n");
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: -8
negative
```

১১ শর্টকাট, সাধারণ ভুল ও সব সংশোধন

পরীক্ষার আগে একবার চোখ বুলাও

⚡ সব শর্টকাট এক জায়গায়

পরিস্থিতি	শর্টকাট
মেশিন, অ্যাসেম্বলি নাকি হাই-লেভেল?	শুধু 0,1 → মেশিন; ADD/SUB/MOV → অ্যাসেম্বলি; if/while/printf → হাই-লেভেল
Compiler নাকি Interpreter?	পুরো প্রোগ্রাম একসাথে + Object code → Compiler (C, C++); লাইন-বাই-লাইন, Object code নেই → Interpreter (Python, BASIC)
প্রজন্ম	1GL মেশিন • 2GL অ্যাসেম্বলি • 3GL হাই-লেভেল (C, Java) • 4GL SQL • 5GL PROLOG/LISP (AI)
প্রতীক	ডিম্ব = শুরু/শেষ • সামান্তরিক = I/O • আয়ত = প্রসেস • হীরা = সিদ্ধান্ত • বৃত্ত = সংযোগ
বৃহত্তম নাকি ক্ষুদ্রতম?	একই কাঠামো; বৃহত্তমে >, ক্ষুদ্রতমে <
লুপ প্রশ্ন	শুধু ৩টা বদলায়: শুরুর মান, step, শেষ সীমা
সিরিজ	১ম পদ = শুরু; ২য় − ১ম = step; বর্গ/ঘন → K*K / K*K*K; যোগে S=0, গুণে S=1
Leap Year	(÷400) OR (÷4 AND ÷100 না)
GCD	R = L%S; L = S; S = R — যতক্ষণ R ≠ 0
while vs do-while	শর্ত আগে → while (০ বার চলতে পারে); body আগে → do-while (≥১ বার)
break vs continue	break = লুপ শেষ; continue = এই ধাপ বাদ
Format specifier	int %d • float %f • char %c • double %lf • string %s
Data size (বোর্ড)	char 1 • int 2 • float 4 • double 8 byte
ভাগ	int/int = int → দশমিক চাইলে 3.0 বা 0.5 লেখো
শিফট	<<n = ×2^n, >>n = ÷2^n
Array	index 0 থেকে size−1
তাপমাত্রা	C/5 = (F−32)/9 = (K−273)/5

❌ সাধারণ ভুল (Common Mistakes)

ভুল	কেন ভুল	সঠিক
scanf-এ & বাদ: `scanf("%d",a);`	& ছাড়া scanf ঠিকানা পায় না — প্রোগ্রাম ক্র্যাশ/ভুল মান	`scanf("%d",&a);`
printf-এ & দেওয়া: `printf("%d",&a);`	মানের বদলে ঠিকানা ছাপে	`printf("%d",a);`
switch-এ break ভুলে যাওয়া	পরের সব case চলে (fall-through)	প্রতিটি case-এর শেষে `break;`
`if(a=5)`	= হলো মান বসানো, তুলনা নয়	`if(a==5)`
`avg=sum/3;` (int sum)	পূর্ণসংখ্যা ভাগে দশমিক হারায়	`sum/3.0`
`(1/2)*b*h`	1/2 = 0	`0.5*b*h`
`9/5*c+32`	9/5 = 1	`9*c/5+32`
গুণফলে S=0 দিয়ে শুরু	0 × যেকোনো = 0	গুণে S=1, যোগে S=0
do-while-এর শেষে ; না দেওয়া	Syntax error	`}while(i<=n);`
for-এ কমা: `for(i=1, i<=n, i++)`	for-এ ঠিক দুটি সেমিকোলন লাগে	`for(i=1; i<=n; i++)`
লুপে i++ ভুলে যাওয়া	অসীম লুপ	প্রতি ধাপে counter আপডেট

চলকের নামে স্পেস/বিন্দু/keyword	নিয়মবিরুদ্ধ	অক্ষর/_ দিয়ে শুরু; A-Z a-z 0-9 _
স্টেটমেন্টের শেষে ; বাদ	Syntax error	`sum=a+b;`
`int A[10]` -এ A[10] ব্যবহার	শেষ index 9	0 থেকে 9
2a, ab লেখা	C-তে গুণ চিহ্ন বাদ চলে না	`2*a` , `a*b`
float-এ abs()	abs() শুধু int	`fabs()`
অ্যালগরিদমে "১০০-এর ছোট" অথচ ১০০ ছাপাতে হবে	শেষ মান বাদ পড়ে	"ছোট বা সমান" (≤)
Compiler ও Interpreter গুলিয়ে ফেলা	কাজের ধরন ভিন্ন	একসাথে বনাম লাইন-বাই-লাইন

⚠ সব সংশোধন — এক নজরে

উৎস	যা ছিল	সঠিক	অংশ
নোট-১	উচ্চস্তরের ভাষা "যন্ত্র নির্ভর"	যন্ত্র **নির্ভর নয়** (Portable)	২
নোট-১	জোড় সংখ্যা: "১০০-এর চেয়ে ছোট হলে ৩ নং ধাপে"	ছোট **বা সমান** (≤)	৮.১১
নোট-১	তিনটির বৃহত্তম: ধাপ-৪-এ "৬ নং ধাপে যাও" নেই	৪ক ও ৪খ উভয় শাখায় "৬ নং ধাপে যাও"	৮.৬
নোট-১	Assembler-এর কার্যপদ্ধতি "ছোট"	কার্যপদ্ধতি **সহজ**	৪
নোট-২	`avg = sum/3;`	`sum/3.0`	১০.২
নোট-২	`area = (1/2)*b*h;`	`0.5*b*h`	১০.৪
নোট-২	হেরনের s = (a+b+c)/3	s = (a+b+c)/**2**	১০.৫
নোট-২	'%' = Modulus বা Absolute value	format-এ % = specifier চিহ্ন; অপারেটর % = ভাগশেষ; Absolute = abs/fabs	৯খ
নোট-২	viod main()	void main()	৯
নোট-২	graphic.h	graphics.h	৯
নোট-২	|x| = abs(x) সব সংখ্যায়	int → abs(), float → fabs()	৯
নোট-২	math.h-এ Cot, Sec, Cosec	নেই — 1/tan, 1/cos, 1/sin	৯
নোট-২	int = 4 byte (এক জায়গায়)	বোর্ডে 2 byte; gcc-তে 4 byte	৯ক
ফাইনাল নোট	অ্যাসেম্বলির অসুবিধা: "লাইন-বাই-লাইন অনুবাদ, প্রতিবার অনুবাদ"	এগুলো Interpreter-এর বৈশিষ্ট্য; Assembler একবারে Object code বানায়	২ক
ফাইনাল নোট	মিড লেভেল: FORTRAN, COBOL	FORTRAN, COBOL = উচ্চস্তরের; মিড লেভেল = C	২ক
ফাইনাল নোট	Interpreter উদাহরণে Pascal	Pascal কম্পাইলড; Interpreter = Python, BASIC	২ক
ফাইনাল নোট	ফ্লোচার্টে নলাকার (Cylinder) = Subroutine	Subroutine = দুই পাশে দাগযুক্ত আয়তক্ষেত্র; Cylinder = Storage/Database	৭
ফাইনাল নোট	double = ৩৭০০০+ অঙ্ক	~১৫–১৬ অঙ্ক নির্ভুলতা, পরিসর ~10^{308}	৯ক
ফাইনাল নোট	Constant উদাহরণ: Pie = 3.1416;	#define PI 3.1416 বা const float PI	৯ক
ফাইনাল নোট	গ্রেড: ≥50 → A-	60–69 = A-, 50–59 = B (পূর্ণ ছক)	৯ঘ

📌 ফাইনাল নোটের সংশোধনগুলোর সাথে মিল

ফাইনাল নোট নিজেও দুটি অমিল ঠিক করেছে — (১) "যন্ত্র নির্ভর নয়" বাক্যটি উচ্চস্তরের ভাষার (অংশ ২-এ আমরাও একই সংশোধন দেখিয়েছি); (২) "গণনা সবসময় ০ থেকে" — আসলে Array index ০ থেকে, কিন্তু লুপ কাউন্টার সমস্যা অনুযায়ী ১ থেকেও শুরু হয়। দুটির সাথেই এই নোট একমত।

১২ অনুশীলন: MCQ + সৃজনশীল (উত্তরসহ)

৩২টি MCQ + ৪টি সৃজনশীল

#	প্রশ্ন ও বিকল্প	উত্তর ও ব্যাখ্যা
1	কম্পিউটার প্রোগ্রামিং-এর প্রবর্তক কে? ক) চার্লস ব্যাবেজ খ) অ্যাডা লাভলেস গ) ডেনিস রিচি ঘ) টিম বার্নার্স-লি	**খ** — Bernoulli Number-এর প্রোগ্রাম লেখেন।
2	কোনটি প্রথম প্রজন্মের (1GL) ভাষা? ক) Assembly খ) C গ) Machine Language ঘ) SQL	**গ** — 1GL = 0 ও 1।
3	অ্যাসেম্বলি ভাষাকে মেশিন ভাষায় অনুবাদ করে — ক) Compiler খ) Interpreter গ) Assembler ঘ) Linker	**গ**
4	কোনটি Compiler-এর বৈশিষ্ট্য নয়? ক) পুরো প্রোগ্রাম একসাথে অনুবাদ খ) Object code তৈরি হয় গ) এক লাইন করে অনুবাদ ঘ) মেমোরি বেশি লাগে	**গ** — লাইন-বাই-লাইন = Interpreter।
5	4GL-এর উদাহরণ — ক) C খ) SQL গ) Assembly ঘ) Prolog	**খ** — ডেটাবেজ-অভিমুখী।
6	কোনটি Volatile মেমোরি? ক) ROM খ) HDD গ) RAM ঘ) SSD	**গ** — বিদ্যুৎ গেলে ডেটা হারায়।
7	প্রোগ্রাম উন্নয়নের সবচেয়ে গুরুত্বপূর্ণ ধাপ — ক) কোডিং খ) প্রোগ্রাম ডিজাইন গ) ডকুমেন্টেশন ঘ) রক্ষণাবেক্ষণ	**খ** — অ্যালগরিদম-ফ্লোচার্ট এই ধাপে।
8	অ্যালগরিদমের চিত্ররূপ — ক) Pseudocode খ) Flowchart গ) Syntax ঘ) Source code	**খ**
9	Flowchart-এ সিদ্ধান্ত বোঝাতে — ক) Oval খ) Rectangle গ) Diamond ঘ) Parallelogram	**গ**
10	Flowchart-এ Input/Output — ক) Oval খ) Parallelogram গ) Circle ঘ) Diamond	**খ**
11	n MOD 2 = 0 হলে সংখ্যাটি — ক) ধনাত্মক খ) ঋণাত্মক গ) জোড় ঘ) বিজোড়	**গ**
12	Leap Year-এর শর্ত — ক) শুধু ÷4 খ) ÷100 গ) ÷400 অথবা (÷4 ও ÷100 না) ঘ) শুধু জোড়	**গ** — 1900 leap নয়, 2000 leap।
13	C-তে char-এর size — ক) 1 byte খ) 2 byte গ) 4 byte ঘ) 8 byte	**ক**
14	scanf("%d",&a); — & বোঝায় — ক) গুণ খ) ঠিকানা (Address) গ) মডুলাস ঘ) মন্তব্য	**খ**
15	while ও do-while-এর প্রধান পার্থক্য — ক) কোনো পার্থক্য নেই খ) do-while কমপক্ষে ১ বার চলে গ) while কমপক্ষে ১ বার চলে ঘ) for-এর সমান	**খ**
16	লুপ থেকে সম্পূর্ণ বের হতে — ক) continue খ) break গ) return ঘ) goto	**খ**
17	00001010 কে <<1 করলে — ক) 5 খ) 10 গ) 20 ঘ) 40	**গ** — বাম শিফট = ×2।
18	বৈধ C চলকের নাম — ক) 1std খ) Std.1 গ) int ঘ) Std_1	**ঘ**
19	A[10]-এর শেষ বৈধ index — ক) 10 খ) 9 গ) 11 ঘ) 0	**খ**
20	switch-এ প্রতিটি case-এর শেষে সাধারণত — ক) continue; খ) break; গ) return; ঘ) goto;	**খ**

21	C ভাষা কে, কবে উদ্ভাবন করেন? ক) জেমস গসলিং, 1995 খ) ডেনিস রিচি, 1972 গ) গুইডো ভ্যান রসাম, 1991 ঘ) বিয়ার্ন স্ট্রাউস্ট্রুপ, 1983	**খ** — Bell Labs।
22	C-কে মিড লেভেল ভাষা বলা হয় কারণ — ক) শুধু 0,1 ব্যবহার করে খ) হার্ডওয়্যার-নিকট কাজ ও Portability দুটোই আছে গ) Interpreter লাগে ঘ) AI ভাষা	**খ**
23	int x = 17 % 5; printf("%d", x); আউটপুট — ক) 3 খ) 2 গ) 3.4 ঘ) 5	**খ** — 17 = 5×3 + 2।
24	float a = 7/2; printf("%.1f", a); আউটপুট — ক) 3.5 খ) 3.0 গ) 4.0 ঘ) 3	**খ** — 7/2 = 3 (int ভাগ) → 3.0।
25	int i = 5, x = i++; এর পর x ও i — ক) 5, 6 খ) 6, 6 গ) 5, 5 ঘ) 6, 5	**ক** — Post: আগে ব্যবহার, পরে বৃদ্ধি।
26	√(x²+y²) এর C রূপ — ক) sqrt(x^2+y^2) খ) sqrt(x*x+y*y) গ) root(x*x+y*y) ঘ) Sqrt(x*x+y*y)	**খ** — ^ C-তে XOR, ঘাত নয়।
27	for(i=1;i<=10;i=i+3) printf("%d ",i); কয়বার চলবে? ক) 3 খ) 4 গ) 10 ঘ) অসীম	**খ** — 1, 4, 7, 10।
28	কোনটি Derived data type? ক) int খ) float গ) Array ঘ) char	**গ**
29	Program রান করার আগে হাতে-কলমে যাচাই — ক) Debugging খ) Dry Run গ) Compiling ঘ) Coding	**খ**
30	প্রোগ্রাম চলে কিন্তু ভুল ফল দেয় — কোন ভুল? ক) Syntax Error খ) Logical Error গ) Data Error নয় ঘ) Compiler Error	**খ**
31	pow(2,3)-এর মান — ক) 6 খ) 8 গ) 9 ঘ) 5	**খ**
32	do{...}while(0); লুপটি কতবার চলবে? ক) 0 খ) 1 গ) অসীম ঘ) Error	**খ** — body আগে চলে।

বহুপদী সমাপ্তিসূচক MCQ

৩৩. Interpreter-এর ক্ষেত্রে — (i) লাইন-বাই-লাইন অনুবাদ করে (ii) Object code তৈরি করে (iii) ভুল সংশোধন সহজ। নিচের কোনটি সঠিক? ক) i ও ii খ) i ও iii গ) ii ও iii ঘ) i, ii ও iii	**খ** — Object code তৈরি করে না।
৩৪. C ভাষায় — (i) প্রতিটি স্টেটমেন্টের শেষে ; দিতে হয় (ii) case sensitive (iii) main() থেকে নির্বাহ শুরু। ক) i ও ii খ) i ও iii গ) ii ও iii ঘ) i, ii ও iii	**ঘ**

সৃজনশীল প্রশ্ন ১ (ফাইনাল নোট)

উদ্দীপক: রাফি একটি C প্রোগ্রাম লিখছে, যেখানে ব্যবহারকারীর কাছ থেকে একটি সংখ্যা নিয়ে তা জোড় না বিজোড় তা যাচাই করতে হবে। এরপর ১ থেকে সেই সংখ্যা পর্যন্ত সবগুলো সংখ্যার যোগফল দেখাতে হবে।

ক) মেশিন ভাষা কী? (১)

কম্পিউটারের নিজস্ব ভাষা, যা শুধু ০ ও ১ দিয়ে গঠিত এবং কম্পিউটার সরাসরি বুঝতে পারে।

খ) "do-while loop কমপক্ষে একবার চলে" — ব্যাখ্যা করো। (২)

do-while-এ প্রথমে body একবার execute হয়, তারপর শর্ত যাচাই হয়। শর্ত সত্য হলে আবার চলে, মিথ্যা হলে থামে। শর্ত যাচাইয়ের আগেই body একবার চলে যায় বলে শর্ত শুরুতেই মিথ্যা হলেও লুপটি কমপক্ষে একবার চলবেই — এটাই while-এর সাথে মূল পার্থক্য।

গ) সংখ্যাটি জোড় না বিজোড় নির্ণয়ের C কোড লেখো। (৩)

উত্তর

```c
#include <stdio.h>
int main()
{
    int a;
    scanf("%d", &a);
    if (a % 2 == 0)
        printf("%d is even", a);
    else
        printf("%d is odd", a);
    return 0;
}
```

ঘ) ১ থেকে সংখ্যাটি পর্যন্ত যোগফল নির্ণয়ের জন্য for loop ব্যবহার করে সম্পূর্ণ C প্রোগ্রাম লেখো। (৪)

উত্তর

```c
#include <stdio.h>
int main()
{
    int n, i, Sum = 0;
    scanf("%d", &n);
    for (i = 1; i <= n; i++)
    {
        Sum = Sum + i;
    }
    printf("Sum=%d", Sum);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 10
Sum=55
```

i=1 থেকে i≤n পর্যন্ত প্রতিবার Sum-এ i যোগ।

সৃজনশীল প্রশ্ন ২ (ফাইনাল নোট)

উদ্দীপক: তামান্না C ভাষায় একটি প্রোগ্রাম লিখছে, যা তিনটি সংখ্যা ইনপুট নিয়ে সর্ববৃহৎটি দেখাবে। সে আরও জানতে চায় কীভাবে দুইটি সংখ্যার গ.সা.গু নির্ণয় করা যায়।

ক) Algorithm কাকে বলে? (১)

কোনো সমস্যা সমাধানের যুক্তিসংগত ও ধারাবাহিক ধাপে ধাপে বর্ণনাকে Algorithm বলে।

খ) "Flowchart তৈরি করা Algorithm-এর চেয়ে কঠিন" — ব্যাখ্যা করো। (২)

Algorithm শব্দে ধাপে ধাপে লেখা যায়। কিন্তু Flowchart-এ প্রতিটি ধাপের জন্য সঠিক প্রতীক (Oval, Rectangle, Diamond…) বাছাই করে আঁকতে হয় এবং তীর দিয়ে যুক্ত করতে হয় — বেশি সময় ও দক্ষতা লাগে এবং পরিবর্তন করাও কঠিন। তাই Flowchart তৈরি তুলনামূলক কঠিন।

গ) তিনটি সংখ্যার মধ্যে সর্ববৃহৎ নির্ণয়ের অ্যালগরিদম লেখো। (৩)

Step1: Start
Step2: Input A, B, C
Step3: A>B সত্য? (ক) হ্যাঁ → Step4 (খ) না → Step5
Step4: A>C সত্য? (ক) হ্যাঁ → Print A, Step6-এ যাও (খ) না → Print C, Step6-এ যাও
Step5: B>C সত্য? (ক) হ্যাঁ → Print B (খ) না → Print C
Step6: End

ঘ) দুইটি সংখ্যার গ.সা.গু নির্ণয়ের সম্পূর্ণ C প্রোগ্রাম লেখো ও বিশ্লেষণ করো। (৪)

উত্তর

```c
#include <stdio.h>
int main()
{
    int L, S, R;
    scanf("%d", &L);
    scanf("%d", &S);
    do
    {
        R = L % S;
        L = S;
        S = R;
    } while (R != 0);
    printf("GCD=%d", L);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 48 18
GCD=6
```

48%18=12 → 18%12=6 → 12%6=0 → GCD 6

সৃজনশীল প্রশ্ন ৩ (সংযোজন)

উদ্দীপক: শিক্ষক বোর্ডে লিখলেন: `S = 1² + 3² + 5² + … + N²`। রনি সমাধানে `int S=0, K;` ঘোষণা করে `for(K=1; K<=N; K++) S=S+K*K;` লিখল।

ক) Bug কী? (১)

প্রোগ্রামের ত্রুটিকে Bug বলে।

খ) "int x = 7/2; এর মান 3" — ব্যাখ্যা করো। (২)

C-তে দুটি int ভাগ করলে ফলও int হয়; দশমিক অংশ (0.5) বাদ পড়ে। তাই 7/2 = 3। দশমিক চাইলে 7.0/2 লিখতে হয়।

গ) উদ্দীপকের ধারাটির অ্যালগরিদম লেখো। (৩)

Step1: Start
Step2: Read N
Step3: S = 0, K = 1
Step4: S = S + K*K
Step5: K = K + 2
Step6: K ≤ N হলে Step4-এ যাও
Step7: Print S
Step8: End

ঘ) রনির কোড ধারাটির সঠিক মান দেবে কি? বিশ্লেষণ করে সঠিক কোড লেখো। (৪)

না। রনি `K++` লিখেছে, ফলে K = 1, 2, 3, 4 … সব সংখ্যার বর্গ যোগ হয় (1²+2²+3²+…)। ধারায় শুধু বিজোড় সংখ্যা, তাই step হবে ২: `K = K + 2`। N = 5 হলে সঠিক উত্তর 1+9+25 = 35, কিন্তু রনির কোডে 55।

সঠিক বনাম রনির কোড

```c
#include <stdio.h>
int main()
{
    int N, K, S = 0, W = 0;
    scanf("%d", &N);
    for (K = 1; K <= N; K = K + 2)
        S = S + K * K;
    for (K = 1; K <= N; K++)
        W = W + K * K;
    printf("Sothik = %d\nRonir = %d", S, W);
    return 0;
}
```

▶ চালিয়ে দেখা আউটপুট (gcc)

```
ইনপুট দিলাম: 5
Sothik = 35
Ronir = 55
```

সৃজনশীল প্রশ্ন ৪ (সংযোজন)

উদ্দীপক: সুমি ১০০ জন শিক্ষার্থীর নম্বর থেকে গ্রেড বের করতে চায়। তার বন্ধু বলল প্রথমে অ্যাসেম্বলি ভাষায় প্রোগ্রাম লিখলে দ্রুত চলবে; সুমি C বেছে নিল।

ক) Interpreter কী? (১)

যে অনুবাদক প্রোগ্রাম উচ্চস্তরের ভাষার প্রোগ্রামকে এক লাইন করে মেশিন ভাষায় অনুবাদ করে সাথে সাথে চালায়।

খ) "C একটি মিড লেভেল ভাষা" — ব্যাখ্যা করো। (২)

C-তে বিটওয়াইজ অপারেটর দিয়ে হার্ডওয়্যারের কাছাকাছি (নিম্নস্তরের মতো) কাজ করা যায়, আবার এটি Portable ও ইংরেজির কাছাকাছি (উচ্চস্তরের মতো)। দুই গুণ একসাথে থাকায় C-কে মিড লেভেল বলা হয়।

গ) সুমির প্রয়োজনে else-if ladder দিয়ে গ্রেড নির্ণয়ের C কোড লেখো। (৩)

অংশ ৯ঘ-এর পূর্ণ গ্রেডিং প্রোগ্রাম (80+ A+, 70 A, 60 A-, 50 B, 40 C, 33 D, নইলে F) লিখবে।

ঘ) সুমির সিদ্ধান্ত যৌক্তিক কিনা — মূল্যায়ন করো। (৪)

অ্যাসেম্বলি দ্রুত চললেও (১) মেশিন-নির্ভর — অন্য কম্পিউটারে চলবে না, (২) লিখতে সময় বেশি ও ভুলের সম্ভাবনা বেশি, (৩) ডিবাগ কঠিন। C-তে (১) if-else, loop, array-এর মতো কাঠামো আছে বলে ১০০ জনের নম্বর লুপ ও অ্যারে দিয়ে সহজে প্রক্রিয়া করা যায়, (২) Portable, (৩) Compiler একবার অনুবাদ করে দ্রুত Object code দেয়। এই ছোট কাজে গতির পার্থক্য নগণ্য, তাই C বেছে নেওয়া **যৌক্তিক**।

১৩ এক পৃষ্ঠার শেষ মুহূর্তের রিভিশন

হলে ঢোকার আগে

প্রোগ্রাম ও ভাষা
প্রোগ্রাম = ধাপে ধাপে নির্দেশাবলি • প্রবর্তক Ada Lovelace • IPO = Input-Process-Output • Bug = ত্রুটি, Debugging = সংশোধন, Dry Run = হাতে যাচাই • ভুল ৩: Data, Logical (চলে কিন্তু ভুল ফল), Syntax (চলেই না)

কম্পিউটার সংগঠন
Input → CPU (ALU + CU + Memory) → Output • RAM = Volatile • ROM/HDD/SSD = Non-volatile

প্রজন্ম
1GL মেশিন (0,1) • 2GL অ্যাসেম্বলি (ADD, SUB; নেমোনিক) • 3GL উচ্চস্তর (C, Java, Python, FORTRAN, COBOL) • 4GL SQL, Oracle • 5GL PROLOG, LISP (AI) • মিড লেভেল = C

মেশিন বনাম উচ্চস্তর
মেশিন: সরাসরি বোঝে, অনুবাদক নেই, দ্রুত, যন্ত্র নির্ভর, Object program • উচ্চস্তর: অনুবাদক লাগে, Portable, সহজ, Source program

অনুবাদক
Assembler: অ্যাসেম্বলি→মেশিন • Compiler: পুরো প্রোগ্রাম একসাথে, Object code, দ্রুত চলে, বেশি মেমোরি (C, C++) • Interpreter: লাইন-বাই-লাইন, Object code নেই, ভুল ধরা সহজ (Python, BASIC)

প্রোগ্রাম তৈরির ৭ ধাপ
নির্দিষ্টকরণ → বিশ্লেষণ → **ডিজাইন** (সবচেয়ে গুরুত্বপূর্ণ) → কোডিং → বাস্তবায়ন (Testing + Debugging) → ডকুমেন্টেশন → রক্ষণাবেক্ষণ

অ্যালগরিদম ও ফ্লোচার্ট
Algorithm = লেখা ধাপ (সহজবোধ্য, সসীম, সার্বজনীন) • Flowchart = চিত্ররূপ • ডিম্ব শুরু/শেষ • সামান্তরিক I/O • আয়ত প্রসেস • হীরা সিদ্ধান্ত • বৃত্ত সংযোগ • দুই-দাগ আয়ত সাবরুটিন • ষড়ভুজ লুপ • ৫ লজিক: Sequential, Decision, Loop, Jump, Subroutine

C ভাষা
Dennis Ritchie, 1972, Bell Labs • main() থেকে শুরু • প্রতি লাইনে ; • case sensitive • গঠন: Documentation → Link → Definition → Global → main → Subprogram

ডেটা টাইপ
char 1 (%c) • int 2 [gcc 4] (%d) • float 4 (%f) • double 8 (%lf) • Built-in / User-defined / Derived (Array, Pointer)

চলকের নাম
অক্ষর/_ দিয়ে শুরু • A-Z a-z 0-9 _ • স্পেস/চিহ্ন/keyword নিষেধ • Std_1 ✓ 1std × Std.1 × int ×

I/O ও অপারেটর
scanf("%d",&a) — & আবশ্যক • printf("%d",a) • \n নতুন লাইন • // ও /* */ কমেন্ট • % = ভাগশেষ • == তুলনা • && || ! • i++ (post) ++i (pre) • << ×2 >> ÷2 • int/int = int

শর্ত ও লুপ
if-else • else-if ladder • nested • switch (প্রতি case-এ break) • while (০ বার সম্ভব) • do-while (≥১ বার, শেষে ;) • for(init; cond; update) • break = বের, continue = টপকানো

ফাংশন, অ্যারে
Library (printf, sqrt) বনাম User-defined • void = কিছু ফেরত দেয় না • int A[10] → index 0–9 • String = char-এর Array, শেষে \0

সূত্র
C/5 = (F−32)/9 = (K−273)/5 • ½bh → 0.5*b*h • হেরন s=(a+b+c)/2 • πr² → 3.1416*r*r • Leap: ÷400 OR (÷4 AND ÷100 না) • GCD: R=L%S, L=S, S=R

সিরিজ
যোগে S=0, গুণে S=1 • শুরু = ১ম পদ • step = ২য় − ১ম • বর্গ K*K, ঘন K*K*K • শর্ত K≤N

🧠 মাস্টার-ট্রিকস
"**মা-অ-উ-অতি-স্বা**" (প্রজন্ম) • "**নি-বি-ডি-কো-বা-ড-র**" (৭ ধাপ) • "**ভোলা RAM**" • "**রিচি-র ৭২-এ বেল**" • "**<< গুণ, >> ভাগ**" • "**Break = দরজা, Continue = টপকানো**" • বৃহত্তম >, ক্ষুদ্রতম <